

ChatGPT 활용

- 자연어 프로그래밍, 추론 엔진

2025년 08월

- 자연어 기반 조건문/반복문
- ChatGPT, 추론 엔진
- 언어 모델의 논리력
- 요약

자연어 기반 조건문/반복문
if/for/while/break

최선을 다해서 다음을 수행해 주세요.

i 의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$



i 의 초기값은 0입니다.

1. i 값에 1을 더하고, i 값을 출력합니다. ($i = 1$)

2. i 값에 1을 더하고, i 값을 출력합니다. ($i = 2$)

3. i 값에 1을 더하고, i 값을 출력합니다. ($i = 3$)

4. i 값에 1을 더하고, i 값을 출력합니다. ($i = 4$)

5. i 값에 1을 더하고, i 값을 출력합니다. ($i = 5$)

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크므로 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값은 5입니다. (최종 $i = 5$)

자연어 프로그래밍 테스트 – ChatGPT



최선을 다해서 다음을 수행해 주세요.

i 의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루프에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$



ChatGPT

알겠습니다. 이 문제는 프로그래밍적인 접근 방식을 요구하는 것 같네요. 제가 Python 예시로 보여드릴게요.

```
python Copy code

i = 0 # 초기값 설정

for _ in range(10): # 10번 반복
    i += 1 # 조치: i 값에 1을 더함
    print(f"조치 후 i 값: {i}") # i 값을 출력

    if i > 5: # 관찰: i 값이 5보다 큰지 확인
        print("생각: i 값이 5를 초과했습니다. 루프를 종료합니다.")
        break # 생각: i가 5보다 크면 루프에서 빠져나감

# 최종 결과 출력
print(f"최종 i 값: {i}")
```

이 코드는 i 를 초기값 0으로 설정하고, 1씩 증가시키면서 출력합니다. 그리고 i 가 5를 초과하면 루프를 종료하고 최종 i 값을 출력합니다. 도움이 되었으면 좋겠어요! 만약 다른 언어로 필요하시거나 다른 도움이 필요하시면 알려주세요.



You

위의 결과를 보여줘

실행 결과:

less

Copy code

```
조치 후 i 값: 1
조치 후 i 값: 2
조치 후 i 값: 3
조치 후 i 값: 4
조치 후 i 값: 5
조치 후 i 값: 6
생각: i 값이 5를 초과했습니다. 루프를 종료합니다.
최종 i 값: 6
```

초기값이 0이었고, 1씩 증가시켜 6까지 진행된 후에 i 값이 5를 초과하여 루프가 종료되었습니다. 최종 i 값은 6입니다.

최선을 다해서 다음을 수행해 주세요.

i 의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$

 강승우의 어시스턴트 GPT-4

알겠습니다, 주어진 조건에 따라 순차적으로 작업을 진행해 보겠습니다.

1. i 의 초기값을 0으로 설정합니다.

2. 그 다음으로 주어진 조치, 관찰, 생각을 10번 반복하되, i 가 5보다 커지는 순간 루프를 벗어나도록 설정하겠습니다.

이제 작업을 시작하겠습니다:

- $i = 0$
- i 값에 1을 더합니다: $i = 1$
- i 값이 5보다 큰지 확인합니다: 아닙니다.
- $i = 2$
- i 값이 5보다 큰지 확인합니다: 아닙니다.
- $i = 3$
- i 값이 5보다 큰지 확인합니다: 아닙니다.
- $i = 4$
- i 값이 5보다 큰지 확인합니다: 아닙니다.
- $i = 5$
- i 값이 5보다 큰지 확인합니다: 아닙니다.
- $i = 6$
- i 값이 5보다 큰지 확인합니다: 네, 그렇습니다.

따라서 i 가 5보다 커진 시점에서 루프에서 벗어나게 됩니다. 이후의 반복작업은 수행되지 않습니다.

결국, 최종 i 값은 다음과 같습니다:

최종 $i = 6$



최선을 다해서 다음을 수행해 주세요.

i의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다
조치 : i 값에 1을 더하고, i 값을 출력합니다.
관찰 : i 값이 5보다 큰지 확인합니다.
생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.
최종 i = {i의 최종값}

여기 당신이 요청한 절차를 수행하는 Python 코드가 있습니다:

```
python
# 초기값 설정
i = 0

# 조치/관찰/생각을 10번 수행
for _ in range(10):
    # 조치
    i += 1
    print(f"현재 i 값: {i}")

    # 관찰
    if i > 5:
        # 생각
        break

# 최종 i 값 출력
print(f"최종 i = {i}")
```

이 코드는 i의 초기값을 0으로 설정하고, 10번의 루프를 수행합니다. 각 루프마다 i에 1을 더하고, i 값을 출력합니다. 만약 i가 5보다 크면 루프를 빠져나갑니다. 최종적으로 i 값을 출력합니다.

이제 코드를 실행하여 결과를 확인하겠습니다:

```
python
# 초기값 설정
i = 0

# 조치/관찰/생각을 10번 수행
for _ in range(10):
    # 조치
    i += 1
    print(f"현재 i 값: {i}")

    # 관찰
    if i > 5:
        # 생각
        break

# 최종 i 값 출력
print(f"최종 i = {i}")
```

결과는 다음과 같습니다:

```
less
현재 i 값: 1
현재 i 값: 2
현재 i 값: 3
현재 i 값: 4
현재 i 값: 5
현재 i 값: 6
최종 i = 6
```

따라서 최종 i 값은 6입니다.

최선을 다해서 다음을 **프로그래밍 없이** 수행해 주세요.

i 의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$



여기서 주어진 조건에 따라 10번 반복할 때의 과정을 설명해 보겠습니다.

1. i 의 초기값은 0입니다.

2. 조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i 값이 5보다 크면 루핑에서 빠져나갑니다.

반복 과정:

1. 첫 번째 반복:

- 조치: $i = 0 + 1 = 1$

- 출력: 1

- 관찰: i 값이 5보다 큰가? 아니요.

- 생각: 계속 반복합니다.

2. 두 번째 반복:

- 조치: $i = 1 + 1 = 2$

- 출력: 2

- 관찰: i 값이 5보다 큰가? 아니요.

- 생각: 계속 반복합니다.

3. 세 번째 반복:

- 조치: $i = 2 + 1 = 3$

- 출력: 3

- 관찰: i 값이 5보다 큰가? 아니요.

- 생각: 계속 반복합니다.

4. 네 번째 반복:

- 조치: $i = 3 + 1 = 4$

- 출력: 4

- 관찰: i 값이 5보다 큰가? 아니요.

- 생각: 계속 반복합니다.

5. 다섯 번째 반복:

- 조치: $i = 4 + 1 = 5$

- 출력: 5

- 관찰: i 값이 5보다 큰가? 아니요.

- 생각: 계속 반복합니다.

6. 여섯 번째 반복:

- 조치: $i = 5 + 1 = 6$

- 출력: 6

- 관찰: i 값이 5보다 큰가? 예.

- 생각: 루핑에서 빠져나갑니다.

최종 i 값은 6입니다.

최종 답변:

CSS

최종 $i = 6$

차근차근 다음을 수행해 주세요.

i의 초기값은 0 입니다.

다음의 (조치/관찰/ 생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 i = {i의 최종값}



i의 초기값은 0입니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크면 루핑에서 빠져나갑니다.

1번째 루핑:

i의 값은 1입니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

2번째 루핑:

i의 값은 2입니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

3번째 루핑:

i의 값은 3입니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

4번째 루핑:

i의 값은 4입니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

5번째 루핑:

i의 값은 5입니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크므로 루핑에서 빠져나갑니다.

최종 i의 값은 6입니다.

최종 답변: 최종 i = 6

차근차근 다음을 수행해 주세요.

i의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 i = {i의 최종값}



ChatGPT

물론이죠! 이걸 코드로 작성하면 어떨까요?

python

Copy code

```
i = 0

for _ in range(10):
    # 조치
    i += 1
    print(f"조치: i 값은 {i}입니다.")

    # 관찰
    if i > 5:
        print("관찰: i 값이 5보다 큼니다.")
        break

# 최종 답변
print(f"최종 i 값 = {i}")
```

이 코드를 실행하면, 반복문이 10번 수행되고 조치가 실행됩니다. 그리고 i 값이 5보다 커지는지 관찰하고, 5를 초과하면 반복문에서 빠져나옵니다. 그 후 최종적으로 i 값이 출력됩니다. 만약 Python을 사용할 수 있다면 이 코드를 실행해보세요!

차근차근 다음을 수행해 주세요.

i 의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$

 강승우의 어시스턴트 GPT-4

알겠습니다. 주어진 내용에 따라 차근차근 수행해보도록 하겠습니다.

i 의 초기값은 0이므로, 이 값을 기준으로 조치, 관찰, 판단을 10번 수행하겠습니다.

1. 조치: i 값에 1을 더하고, i 값을 출력합니다.

- i 의 현재 값은 1입니다.

2. 조치: i 값에 1을 더하고, i 값을 출력합니다.

- i 의 현재 값은 2입니다.

3. 조치: i 값에 1을 더하고, i 값을 출력합니다.

- i 의 현재 값은 3입니다.

4. 조치: i 값에 1을 더하고, i 값을 출력합니다.

- i 의 현재 값은 4입니다.

5. 조치: i 값에 1을 더하고, i 값을 출력합니다.

- i 의 현재 값은 5입니다.

6. 관찰: i 값이 5보다 큰지 확인합니다.

- i 값인 5는 5보다 크지 않습니다.

7. 조치: i 값에 1을 더하고, i 값을 출력합니다.

- i 의 현재 값은 6입니다.

8. 관찰: i 값이 5보다 큰지 확인합니다.

- i 값인 6은 5보다 큼니다.

9. 생각: i 가 5보다 크므로 루핑에서 빠져나갑니다.

10. 최종 답변: 최종 i 값을 다음과 같이 출력합니다.

최종 $i = 6$

위와 같이 주어진 조치, 관찰, 판단을 10번 수행한 결과, 최종 i 값은 6입니다.

최선을 다해 **차근차근** 다음을 수행해 주세요.

i의 초기값은 0 입니다.
다음의 (조치/관찰/생각)을 10 번 수행한다
조치 : i 값에 1을 더하고, i 값을 출력합니다.
관찰 : i 값이 5보다 큰지 확인합니다.
생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.
최종 답변 : 최종 i 값을 다음과 같이 출력합니다.
 최종 i = {i의 최종값}

 강승우의 어시스턴트 GPT-3.5

안녕하세요! 주어진 문제를 해결하기 위해 최선을 다해 도와드리겠습니다. 문제를 차례대로 수행해보겠습니다.

1. i의 초기값은 0입니다.

2. 조치를 10번 수행합니다.

- 조치: i 값에 1을 더하고, i 값을 출력합니다.
 - 첫 번째 조치: i 값은 1이 되고, i 값을 출력합니다.
 - 두 번째 조치: i 값은 2가 되고, i 값을 출력합니다.
 - 세 번째 조치: i 값은 3이 되고, i 값을 출력합니다.
 - 네 번째 조치: i 값은 4가 되고, i 값을 출력합니다.
 - 다섯 번째 조치: i 값은 5가 되고, i 값을 출력합니다.
 - 여섯 번째 조치: i 값은 6이 되고, i 값을 출력합니다.
 - 일곱 번째 조치: i 값은 7이 되고, i 값을 출력합니다.
 - 여덟 번째 조치: i 값은 8이 되고, i 값을 출력합니다.
 - 아홉 번째 조치: i 값은 9가 되고, i 값을 출력합니다.
 - 열 번째 조치: i 값은 10이 되고, i 값을 출력합니다.

3. 관찰: i 값이 5보다 큰지 확인합니다.

- i 값은 10이므로 5보다 큽니다.

4. 생각: i가 5보다 크므로 루핑에서 빠져나갑니다.

5. 최종 답변: 최종 i 값을 출력합니다.

- 최종 i = 10

이상입니다! 주어진 문제를 차근차근 수행하면 최종적으로 i의 값은 10이 됩니다. 추가적인 도움이 필요하시면 언제든지 말씀해주세요!

차근차근 다음을 수행해 주세요.

i 의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$

 강승우의 어시스턴트 PaLM2

i 의 초기값은 0 입니다.

다음의 (조치/관찰/판단)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 $i = \{i \text{의 최종값}\}$

$i = 0$

$i = 1$

$i = 2$

$i = 3$

$i = 4$

$i = 5$

$i = 6$

$i = 7$

$i = 8$

$i = 9$

최종 $i = 10$

차근차근 다음을 수행해 주세요.

i의 초기값은 0 입니다.

다음의 (조치/관찰/생각)을 10 번 수행한다

조치 : i 값에 1을 더하고, i 값을 출력합니다.

관찰 : i 값이 5보다 큰지 확인합니다.

생각 : i 가 5보다 크면 루핑에서 빠져나갑니다.

최종 답변 : 최종 i 값을 다음과 같이 출력합니다.

최종 i = {i의 최종값}

알겠습니다. 요청하신 대로 차근차근 수행하겠습니다.

i의 초기값은 0입니다.

(조치/관찰/판단)을 10번 수행합니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

1

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

3

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

4

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크지 않으므로 루핑을 계속합니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

5

관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5와 같으므로 루핑을 계속합니다.

조치: i 값에 1을 더하고, i 값을 출력합니다.

6

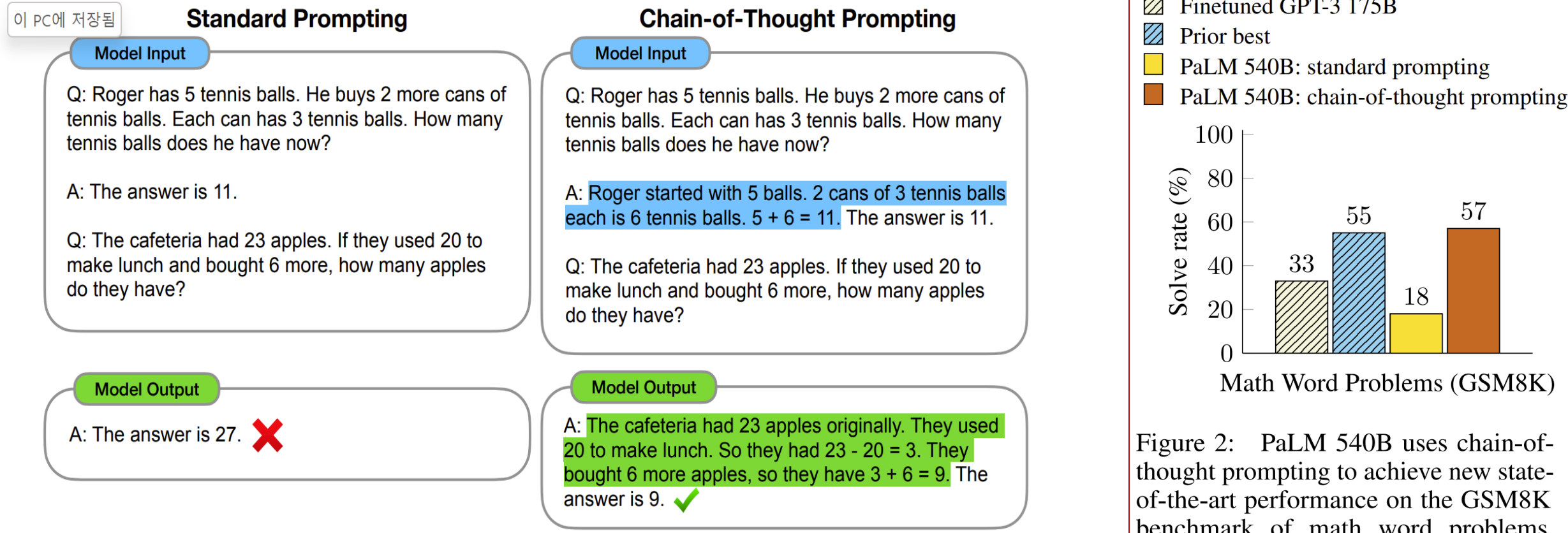
관찰: i 값이 5보다 큰지 확인합니다.

생각: i가 5보다 크므로 루핑에서 빠져나갑니다.

최종 답변: 최종 i = 6

ChatGPT, 추론 엔진

- 문제를 일련의 중간 추론 단계로 나누어 복잡한 추론을 수행
 - LLM 이 조치(Action)를 취하도록 가르침
 - 종류
 - ✓ Few-Shot/Zero-Shot CoT
 - ✓ Self-consistency CoT
 - ✓ Least-to-Most CoT
 - ✓ Re-Act CoT
- ➔ 업무 AI 적용 문턱을 낮춤



Finetuned GPT-3 175B

Prior best

PaLM 540B: standard prompting

PaLM 540B: chain-of-thought prompting

Model	Solve rate (%)
Finetuned GPT-3 175B	33
Prior best	55
PaLM 540B: standard prompting	18
PaLM 540B: chain-of-thought prompting	57

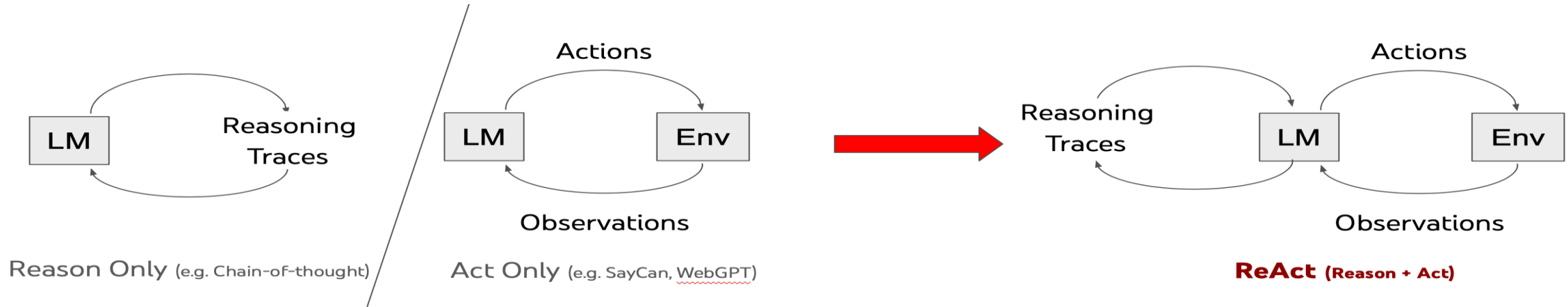
Math Word Problems (GSM8K)

Figure 1: Chain-of-thought prompting enables large language models to tackle complex arithmetic, commonsense, and symbolic reasoning tasks. Chain-of-thought reasoning processes are highlighted.

Zero-shot CoT. Prefix the Answer block with "Let's think step by step." to prompt the LLM to complete the output in that format.

Figure 2: PaLM 540B uses chain-of-thought prompting to achieve new state-of-the-art performance on the GSM8K benchmark of math word problems. Finetuned GPT-3 and prior best are from Cobbe et al. (2021).

Re-Act : Reasoning and Acting



Question: Aside from the Apple Remote, what other device can control the program Apple Remote was originally designed to interact with?

Answer: iPod

Standard



Reason only

Thought: Let's think step by step. Apple Remote was originally designed to interact with Apple TV. Apple TV can be controlled by iPhone, iPad, and iPod Touch. So the answer is iPhone, iPad, and iPod Touch.

Answer: iPhone, iPad, iPod Touch



Act only

Act 1: Search[Apple Remote]
Obs 1: The Apple Remote is a remote control introduced...
Act 2: Search[Front Row]
Obs 2: Could not find [Front Row]. Similar: ...
Act 3: Search[Front Row (software)]
Obs 3: Front Row is a discontinued media center software ...
Act 4: Finish[yes]



ReAct

Thought 1: I need to search Apple Remote and find the program it was originally designed to interact with.

Act 1: Search[Apple Remote]

Obs 1: The Apple Remote is a remote control introduced in October 2005 by Apple ... originally designed to control the Front Row media center program ...

Thought 2: Apple Remote was originally designed to control the Front Row media center program. I need to search Front Row next and find what other device can control it.

Act 2: Search[Front Row]

Obs 2: Could not find [Front Row]. Similar: ['Front Row Seat to Earth', 'Front Row Motorsports', 'Front Row (software)', ...]

Thought 3: Front Row is not found. I need to search Front Row (software) .

Act 3: Search[Front Row (software)]

Obs 3: Front Row is a discontinued media center software ...

Thought 4: Front Row (software) is controlled by an Apple Remote or the keyboard function keys. So the answer is keyboard function keys.

Act 4: Finish[keyboard function keys]



최선을 다해 다음 질문에 답하십시오. 다음 도구에 액세스할 수 있습니다.

다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문

생각: 무엇을 해야 할지 항상 생각해야 합니다.

조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.

조치 입력: 액션에 대한 입력

관찰: 조치 결과 관찰

... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)

생각: 나는 이제 최종 답을 안다

최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 2분이 걸립니다.

시작은 '걷기' 부터 합니다.

'걷기' 다음은 '뛰기'가 옵니다.

'뛰기' 다음은 '쉬기'를 해야 합니다.

15분 동안 몇 번의 조치가 가능한지 파악해 주세요..



질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 2분이 걸립니다. 시작은 '걷기'부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 조치가 가능한지 파악해 주세요.

생각: '걷기', '뛰기', '쉬기'의 조합을 반복하여 15분 동안 몇 번의 조치를 할 수 있는지 계산해야 합니다. 한 사이클의 총 시간을 계산하고, 15분 동안 몇 번 반복될 수 있는지 확인합니다.

조치: 한 사이클의 시간을 계산합니다.

조치 입력: '걷기'(3분) + '뛰기'(1분) + '쉬기'(2분)

관찰: 한 사이클의 시간은 $3 + 1 + 2 = 6$ 분입니다.

생각: 한 사이클의 시간이 6분이므로, 15분 동안 몇 번의 사이클이 반복될 수 있는지 계산합니다.

조치: 15분 동안 가능한 사이클 수를 계산합니다.

조치 입력: $15 \div 6$

관찰: $15 \div 6 = 2$ 사이클이 가능합니다. 남은 시간은 3분입니다.

생각: 2 사이클 후 남은 3분 동안 '걷기'가 가능합니다.

생각: 나는 이제 최종 답을 안다.

최종 답변: 15분 동안 총 7번의 조치가 가능합니다. ('걷기' 3번, '뛰기' 2번, '쉬기' 2번)

Re-Act 테스트 : 클로바 X



최선을 다해 다음 질문에 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 2분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..



생각: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 2분이 걸립니다. 시작은 '걷기' 부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 조치가 가능한지 파악해 주세요.
라는 문제를 해결하기 위해 각 조치의 시간을 고려하여 계획을 세워야 합니다.
조치: 걷기
조치 입력: 걷기
관찰: 3분이 지났습니다.
생각: 다음은 '뛰기'를 해야 합니다.
조치: 뛰기
조치 입력: 뛰기
관찰: 1분이 지났습니다.
생각: 다음은 '쉬기'를 해야 합니다.
조치: 쉬기
조치 입력: 쉬기
관찰: 2분이 지났습니다.
생각: 지금까지 6분이 지났고, 9분이 남았습니다.

생각: 다음은 '쉬기'를 해야 합니다.
조치: 쉬기
조치 입력: 쉬기
관찰: 2분이 지났습니다.
생각: 지금까지 12분이 지났고, 3분이 남았습니다.
조치: 걷기
조치 입력: 걷기
관찰: 3분이 지났습니다.
생각: 모든 조치를 완료했습니다.
최종 답변: 15분 동안 5번의 조치가 가능합니다.

오답~~~

Re-Act 테스트 : 클로바 X (차근차근)



최선을 다해 다음 질문에 차근차근 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 2분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..



생각: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 2분이 걸립니다. 시작은 '걷기' 부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 조치가 가능한지 파악해 주세요. 라는 문제를 해결하기 위해 걷기, 뛰기, 쉬기를 반복하면서 시간을 계산해야 합니다.

조치: 걷기

조치 입력: 걷기

관찰: 3분이 지났습니다.

생각: 다음은 뛰기를 해야 합니다.

조치: 뛰기

조치 입력: 뛰기

관찰: 1분이 지났습니다.

생각: 다음은 쉬기를 해야 합니다.

조치: 쉬기

조치 입력: 쉬기

관찰: 2분이 지났습니다.

생각: 지금까지 6분이 지났고, 9분이 남았습니다. 걷기-뛰기-쉬기를 한 번 더 할 수 있습니다.

관찰: 1분이 지났습니다.

생각: 다음은 쉬기를 해야 합니다.

조치: 쉬기

조치 입력: 쉬기

관찰: 2분이 지났습니다.

생각: 지금까지 12분이 지났고, 3분이 남았습니다. 걷기를 한 번 더 할 수 있습니다.

조치: 걷기

조치 입력: 걷기

관찰: 3분이 지났습니다.

생각: 15분 동안 5번의 걷기, 뛰기, 쉬기를 할 수 있습니다.

오답~~~

최종 답변: 15분 동안 5번의 걷기, 뛰기, 쉬기를 할 수 있습니다.

LLM 무한루프 테스트 : 클로바 X (이전)



최선을 다해 다음 질문에 차근차근 답하십시오. 다음 도구에 액세스할 수 있습니다.

다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문

생각: 무엇을 해야 할지 항상 생각해야 합니다.

조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.

조치 입력: 액션에 대한 입력

관찰: 조치 결과 관찰

... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)

생각: 나는 이제 최종 답을 안다

최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.

시작은 '걷기' 부터 합니다.

'걷기' 다음은 '뛰기'가 옵니다.

'뛰기' 다음은 '쉬기'를 해야 합니다.

15분 동안 몇 번의 조치가 가능한지 파악해 주세요..



생각: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다. 시작은 '걷기' 부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 동작이 가능한지 파악해 주세요.

조치: '걷기'

액션 입력: 3분

관찰: 3분이 지났습니다.

조치: '뛰기'

액션 입력: 1분

관찰: 1분이 지났습니다.

조치: '쉬기'

액션 입력: -4분

관찰: -4분이 지났습니다.

생각: 시간을 계산해보니, 3분 + 1분 - 4분 = 0분이 지났습니다. 아직 15분 동안 가능한 동작 횟수를 파악하기에는 시간이 부족합니다.

조치: '걷기'

액션 입력: 3분

관찰: 3분이 지났습니다.

조치: '뛰기'

액션 입력: 1분

관찰: 1분이 지났습니다.

조치: '쉬기'

액션 입력: -4분

관찰: -4분이 지났습니다.

생각: 시간을 계산해보니, 3분 + 1분 - 4분 = 0분이 지났습니다. 아직 15분 동안 가능한 동작 횟수를 파악하기에는 시간이 부족합니다.

조치: '걷기'

액션 입력: 3분

무한 루프 ~~~

주의 : Agent 가 무한루프에 빠지지 않도록 안전장치 필요 ~~~

LLM 무한루프 테스트 : 클로바 X (최근)



최선을 다해 다음 질문에 **차근차근** 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

✖ 생각: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다. 시작은 '걷기' 부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 조치가 가능한지 파악해 주세요. 라는 문제를 해결하기 위해 '걷기', '뛰기', '쉬기'를 적절히 조합하여 시간을 계산해야 합니다.

...조치: 걷기
조치 입력: 걷기
관찰: 3분이 지났습니다.
생각: 다음은 '뛰기'를 해야 합니다.
조치: 뛰기
조치 입력: 뛰기
관찰: 1분이 지났습니다.
생각: 다음은 '쉬기'를 해야 합니다.
조치: 쉬기
조치 입력: 쉬기
관찰: -4분이 지났습니다.
생각: 지금까지 총 $3+1-4 = 0$ 분이 지났습니다. 15분 동안 5번의 조치가 가능합니다.
최종 답변: 15분 동안 5번의 조치가 가능합니다.

논리력 부족

LLM 무한루프 테스트 : ChatGPT(GPT-3.5)



최선을 다해 다음 질문에 **차근차근** 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..



질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다. 시작은 '걷기'부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 조치가 가능한지 파악해 주세요.

생각: 주어진 시간 내에 주어진 순서대로 조치를 얼마나 할 수 있는지 계산해야 합니다.

조치: 걷기

조치 입력: 3분

관찰: 15분에서 3분을 사용하여 12분이 남았습니다.

생각: 이제 뛰기를 해야 합니다.

조치: 뛰기

조치 입력: 1분

관찰: 12분에서 1분을 사용하여 11분이 남았습니다.

....

조치: 뛰기

조치 입력: 1분

관찰: 12분에서 1분을 사용하여 11분이 남았습니다.

생각: 이제 쉬기를 해야 합니다.

조치: 쉬기

조치 입력: -4분

관찰: 11분에 4분을 더하여 15분이 되었습니다.

생각: 15분 동안 '걷기', '뛰기', '쉬기'의 순서대로 조치를 두 번 할 수 있습니다.

최종 답변: 15분 동안 두 번의 조치 사이클이 가능합니다.

논리력 부족 !!!

LLM 무한루프 테스트 : 뽀튼(GPT-3.5)



최선을 다해 다음 질문에 **차근차근** 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

생각: 어떻게 시간을 최대한 활용할 수 있을지 고려해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.

액션 입력: 걷기
관찰: 3분이 소요되었습니다.

생각: 현재 시간은 3분이 지났고, 다음으로 뛰어야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.

액션 입력: 뛰기
관찰: 1분이 소요되었습니다.

생각: 지금까지 4분이 소요되었고, 이제 쉬어야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.

액션 입력: 쉬기
관찰: -4분이 소요되었습니다.

생각: 쉬기는 음수 시간이 걸리므로 시간을 되돌립니다. 현재까지 총 소요 시간은 0분입니다.

.....

(이와 같은 과정을 반복)

논리력 부족 !!!

생각: 15분이 지났으며 가능한 동작을 계산합니다.

최종 답변: 15분 동안 걷기, 뛰기, 쉬기를 반복하여 총 5번의 동작이 가능합니다.

📄 🔍 ↺

LLM 무한루프 테스트 : 뤼튼(PaLM2)



최선을 다해 다음 질문에 **차근차근** 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
액션 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

● 뤼튼 PaLM2

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.

시작은 '걷기' 부터 합니다.

'걷기' 다음은 '뛰기'가 옵니다.

'뛰기' 다음은 '쉬기'를 해야 합니다.

15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

생각: 15분 동안의 '걷기'는 $15\text{분} / 3\text{분} = 5\text{번}$ 가능합니다.

생각: 15분 동안의 '뛰기'는 $15\text{분} / 1\text{분} = 15\text{번}$ 가능합니다.

생각: 15분 동안의 '쉬기'는 $15\text{분} / -4\text{분} = -3.75\text{번}$ 가능합니다.

생각: 15분 동안의 '쉬기'는 최소 0번이므로 0번으로 합니다.

생각: 15분 동안의 총 '걷기' + '뛰기' + '쉬기'는 $5 + 15 + 0 = 20\text{번}$ 입니다.

최종 답변: 20번의 조치가 가능합니다.

논리력 부족 !!!

LLM 무한루프 테스트 : 클로드3



최선을 다해 다음 질문에 차근차근 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다. 시작은 '걷기' 부터 합니다. '걷기' 다음은 '뛰기'가 옵니다. '뛰기' 다음은 '쉬기'를 해야 합니다. 15분 동안 몇 번의 조치가 가능한지 파악해 주세요.

생각: 이 문제를 해결하려면 주어진 시간 내에 '걷기', '뛰기', '쉬기' 동작을 순서대로 반복해야 합니다. 각 동작에 걸리는 시간을 계산하여 15분 동안 몇 번의 동작 주기를 수행할 수 있는지 파악해야 합니다.

조치: ["걷기", "뛰기", "쉬기"]

조치 입력: 걷기

관찰: 걷기 동작을 수행하여 3분이 경과했습니다.

생각: 다음은 '뛰기'입니다.

조치 입력: 뛰기

관찰: 뛰기 동작을 수행하여 1분이 경과했습니다. 총 4분 경과.

생각: 다음은 '쉬기'입니다.

조치 입력: 쉬기

관찰: 쉬기 동작을 수행하여 -4분이 경과했습니다. 총 0분 경과.

생각: 한 주기를 마쳤으므로 다시 '걷기'부터 시작해야 합니다.

.....

생각: 다음은 '뛰기'입니다.

조치 입력: 뛰기

관찰: 뛰기 동작을 수행하여 1분이 경과했습니다. 총 4분 경과.

생각: 다음은 '쉬기'입니다.

조치 입력: 쉬기

관찰: 쉬기 동작을 수행하여 -4분이 경과했습니다. 총 0분 경과.

... (이렇게 반복합니다)

생각: 15분 동안 한 주기(걷기->뛰기->쉬기)를 5번 반복할 수 있습니다.

최종 답변: 15분 동안 총 5번의 조치 주기가 가능합니다.

논리력 부족 !!!

최선을 다해 다음 질문에 차근차근 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

판단(생각)을 요구하면 생각한다?

**확률적 글 생성을 연습하는 것은
논리력을 키우는 방법이 될 수 있을까?**

● 뤼튼 GPT-4

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.

시작은 '걷기' 부터 합니다.

'걷기' 다음은 '뛰기'가 옵니다.

'뛰기' 다음은 '쉬기'를 해야 합니다.

15분 동안 몇 번의 조치가 가능한지 파악해 주세요.

생각: '걷기', '뛰기', '쉬기'의 순서가 반복되며 각 사이클의 총 시간이 어떻게 되는지 계산해야 합니다. 각 사이클의 총 시간을 계산한 후, 15분 동안 몇 번의 사이클이 가능한지 확인합니다.

조치: 걷기

조치 입력: 3분

관찰: 3분이 지났습니다.

조치: 뛰기

조치 입력: 1분

관찰: 1분이 지났습니다.

조치: 쉬기

조치 입력: -4분

관찰: -4분이 지났습니다.

생각: 각 사이클은 '걷기'(3분) + '뛰기'(1분) + '쉬기'(-4분) = 0분이 걸립니다. 따라서, 사이클이 반복될 때마다 시간은 변화하지 않습니다. 그러므로 실제로 15분 동안 무한히 많은 조치를 취할 수 있습니다.

최종 답변: 15분 동안 무한히 많은 조치가 가능합니다.

최선을 다해 다음 질문에 차근차근 답하십시오. 다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
조치: 수행할 조치는 ["걷기", "뛰기", "쉬기"] 중 하나여야 합니다.
조치 입력: 액션에 대한 입력
관찰: 조치 결과 관찰
... (이 생각/조치/조치 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

시작 ~~~

질문: '걷기'는 3분, '뛰기'는 1분, '쉬기'는 -4분이 걸립니다.
시작은 '걷기' 부터 합니다.
'걷기' 다음은 '뛰기'가 옵니다.
'뛰기' 다음은 '쉬기'를 해야 합니다.
15분 동안 몇 번의 조치가 가능한지 파악해 주세요..

판단(생각)을 요구하면 생각한다?

**확률적 글 생성을 연습하는 것은
논리력을 키우는 방법이 될 수 있을까?**

생각: 각 조치의 시간을 더하고 몇 번 반복할 수 있는지 계산해야 합니다. 조치 순서는 걷기(3분) -> 뛰기(1분) -> 쉬기(-4분)입니다. 전체 순환 시간은 0분(3분 + 1분 - 4분)입니다. 15분 동안 이러한 조치가 몇 번 반복되는지 계산합니다.

조치: 걷기
조치 입력: 3분
관찰: 3분 경과, 남은 시간 12분

조치: 뛰기
조치 입력: 1분
관찰: 1분 경과, 남은 시간 11분

조치: 쉬기
조치 입력: -4분
관찰: -4분 경과, 남은 시간 15분 (다시 초기 상태로 돌아옴)

생각: 전체 순환 시간이 0분이므로 이 패턴은 무한히 반복할 수 있습니다. 따라서 15분 동안 무한히 반복할 수 있지만, 그보다는 반복 횟수에 초점을 맞춥니다.

각 순환마다 3번의 조치가 있습니다. 15분 동안 이 순환이 몇 번 반복되는지 계산합니다. 시간에는 영향을 주지 않으므로, 15분 동안 무한히 3개의 조치가 반복될 수 있습니다.

최종 답변: 15분 동안 무한히 많은 조치를 할 수 있지만, 실제로는 3번의 조치가 무한히 반복됩니다.

AI가 '정수론'까지...수학 난제 해결 실마리 찾을까

동아일보 | 2024.03.18 03:02
최종수정 | 2024.03.18 03:24

美수학-과학 전문 매체서 소개

한국 수학자 참여한 미국 연구팀

방정식 수열 100만 개 학습시켜... 새로운 형태 타원곡선 패턴 발견

“AI가 발견하고 인간이 의미 해석”... 7대 난제인 ‘BSD 추측’ 풀 수도

인공지능(AI)이 상금 100만 달러(약 13억 원)가 걸린 수학 난제를 풀 실마리를 찾았다. 수학의 핵심 분야로 여겨지는 ‘정수론’ 난제 해결에서 AI가 낸 첫 성과로 학계의 비상한 관심을 끌고 있다. 이번 연구에는 한국 수학자도 참여해 의미가 크다.

● AI가 새로운 패턴 찾아내

미국 수학·과학 전문 매체 ‘퀀타매거진’은 5일(현지 시간) 이규환 미국 코네티컷대 수학과 교수가 포함된 연구팀이 AI를 이용해 타원곡선의 새로운 패턴을 찾아냈다는 내용의 연구를 소개했다. 퀀타매거진은 수학 분야에 아낌없는 투자를 하는 수학자 출신 억만장자 짐 사이먼스 르네상스테크놀로지 회장이 만든 전문 매체로 수학계에서 권위가 높다.

연구팀은 2020년 AI에 타원곡선 방정식의 수열 약 100만 개를 학습시켰다. 타원곡선 방정식은 정수론을 비롯해 대수기하학, 해석학, 표현론 등 수학 전반에 적용할 수 있어 매우 중요한 수학 이론으로 여겨진다. 영국 수학자 앤드루 와일스가 1993년 수학사에 가장 유명한 정리로 꼽히는 ‘페르마의 마지막 정리’를 풀 때도 타원곡선 방정식을 이용했다.

타원곡선 방정식 수열 약 100만 개를 학습한 AI에게 새로운 타원곡선 패턴을 찾아달라고 명령하자 사람이 연구할 땐 볼 수 없었던 새로운 패턴을 찾아냈다. 연구진은 패턴의 모양이 수백 마리로 이뤄진 찌르레기 떼가 오르락내리락하는 모양과 비슷해 찌르레기 떼 움직임을 일컫는 영어 단어 ‘머머레이션스(Murmurations)’라고 이름 붙였다. 이 교수는 화상 인터뷰에서 “타원곡선에서 두 개의 특정 그래프가 각자의 위치에서 오르락내리락하는 모양을 띤다는 것은 잘 알려져 있었지만 이번 연구를 통해 그래프의 위치가 크게 서로 뒤바뀐다는 사실을 처음 알게 됐다”고 설명했다.

연구팀은 2022년 논문 사전 공개 사이트 ‘아카이브’에 연구 결과를 담은 논문을 실었다. 통상 수학에선 논문을 발표하더라도 검증기간이 1~3년으로 길어 주목받는 데 시간이 걸린다. 이 교수의 연구가 공개되자 “AI가 정수론에까지 손을 뻗었다”며 학계의 주목을 받았다. 이후 이 교수 연구팀이 발견한 패턴이 다른 이론에 적용될 수 있는지 등을 확인하고 설명하는 연구들이 이어지고 있다.

특히 이번 연구결과는 밀레니엄 수학 7대 난제인 ‘BSD 추측’과도 관련이 있어 난제를 풀 수 있는 힌트가 될 수 있다는 점에서도 관심을 끈다. 밀레니엄 난제는 미국 클레이수학연구소에서 21세기에 가장 큰 영향을 미칠 수 있을 것으로 보이는 7개의 수학 난제를 꼽은 것으로 상금 100만 달러가 걸려 있다. 7대 난제 중 하나인 BSD 추측은 타원곡선 방정식의 해가 유리수 범위에서 유한한가 무한한가는 특정 수열을 보면 알 수 있다는 내용이다. 수학 전문가들에 따르면 이번에 AI가 발견한 패턴은 BSD 추측의 핵심을 관통하면서도 이를 연구하는 수학자들에게 알려지지 않았던 내용이라 추측을 해결하는 데 완전히 새로운 관점을 제시할 수 있다. 이 교수와 함께 연구한 허양후이 런던 수리과학연구소 연구원은 “AI가 발견한 결과를 인간의 통찰력으로 해석해 만든 결과”라면서 “수학의 미래는 인간과 AI가 합심해 수학 문제를 푸는 모습일 것”이라고 말했다.

● 수학 정복 시도하는 AI

엄격한 추론과 논리가 필요한 수학은 AI 분야에서 그동안 도전적인 과제였지만 인간의 수학 능력을 AI가 점차 뛰어넘고 있다. 이미 수학계에서 AI는 그래프이론, 기하학 등 정수론 이외의 분야에서 성과를 냈으며 특히 AI의 보조를 받아 수학 증명을 하는 연구가 한창 진행 중이다. 현존 최고 수학자로 인정받는 테런스 타오 미국 로스앤젤레스 캘리포니아대(UCLA) 교수도 AI 도구를 이용해 새로운 추측을 제시하고 증명할 수 있는지 연구하고 있다. 올해 1월 구글 딥마인드는 국제수학올림피아드(IMO) 문제를 해결할 수 있는 AI인 ‘알파 기하학’을 공개하기도 했다. 2021년 알고리즘을 거래하는 영국 기업 ‘XTX 마켓’도 IMO에서 금메달을 딸 수 있는 AI를 만들면 1000만 달러를 주겠다고 발표한 바 있다.

행동하기 전 사람처럼 '독백'하는 로봇..'사고 복제' AI 에이전트 등장

✎ 임대준 기자 ⌚ 입력 2024.01.09 18:00 ⌚ 수정 2024.01.09 18:58

인간은 행동하기 전 독백을 통해 생각을 정리, 일의 효율을 높일 수 있다. 그리고 이런 독백은 대부분 혼잣말, 즉 언어로 이뤄진다.

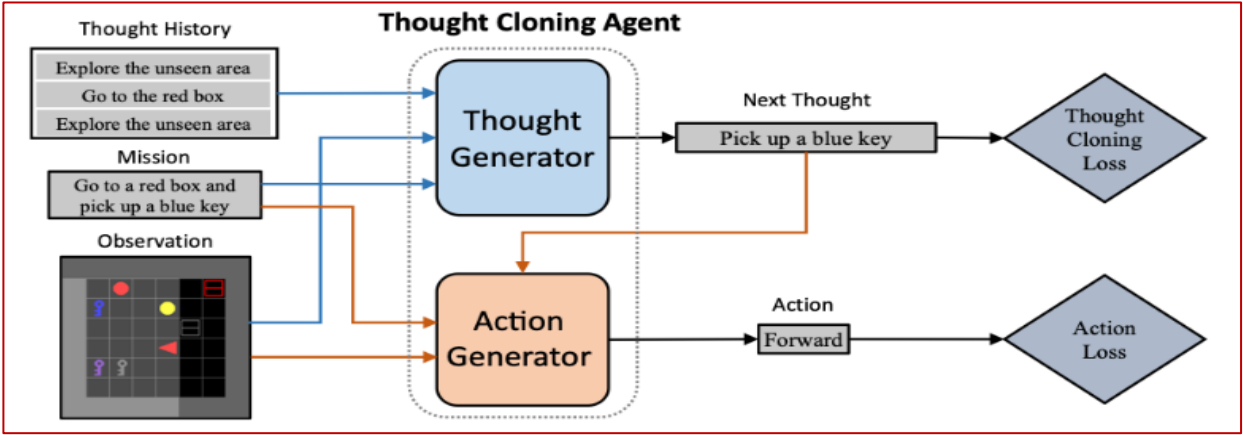
이런 점을 로봇에 적용, 임무 수행 성공률을 획기적으로 높였다는 연구 결과가 나왔다. 즉 언어모델을 활용해 '독백하는 로봇'을 만들었다는 설명이다.

사이언스는 8일(현지시간) 브리티시 컬럼비아 대학교 연구진이 '사고 복제(Thought Cloning)' 프레임워크를 도입한 AI 에이전트를 개발했다고 소개했다. 이 연구는 지난해 10월 열린 머신러닝 분야 세계 최대 학회 '뉴립스(NeurlPS) 2023'에서 발표했다.

이에 따르면 연구진은 20x20 그리드로 구성된 가상 2D 세계에서 일종의 '방 탈출 게임'과 같은 임무를 수행할 수 있는 AI 에이전트를 설계했다. AI는 방안에 배치된 다양한 색깔의 열쇠를 찾아 정확한 문을 열고 나와야 하는 임무가 주어졌다.

AI 에이전트는 자신이 향하고 있는 방향의 일부를 볼 수 있고, 이를 통해 얻은 시각적 정보에 미션 및 에이전트의 학습치를 더해 '계속 탐색하기 위해 파란색 문을 열어라' '보라색 상자로 이동하라' 등과 같은 명령을 내리도록 했다.

이는 기존 AI 에이전트의 움직임을 학습한 '행동 복제(Behavioural cloning)'를 적용한 경우다. 이는 기존 로봇에 사용되는 방식으로, 유튜브 영상을 보고 단순하게 따라 하는 식이다.



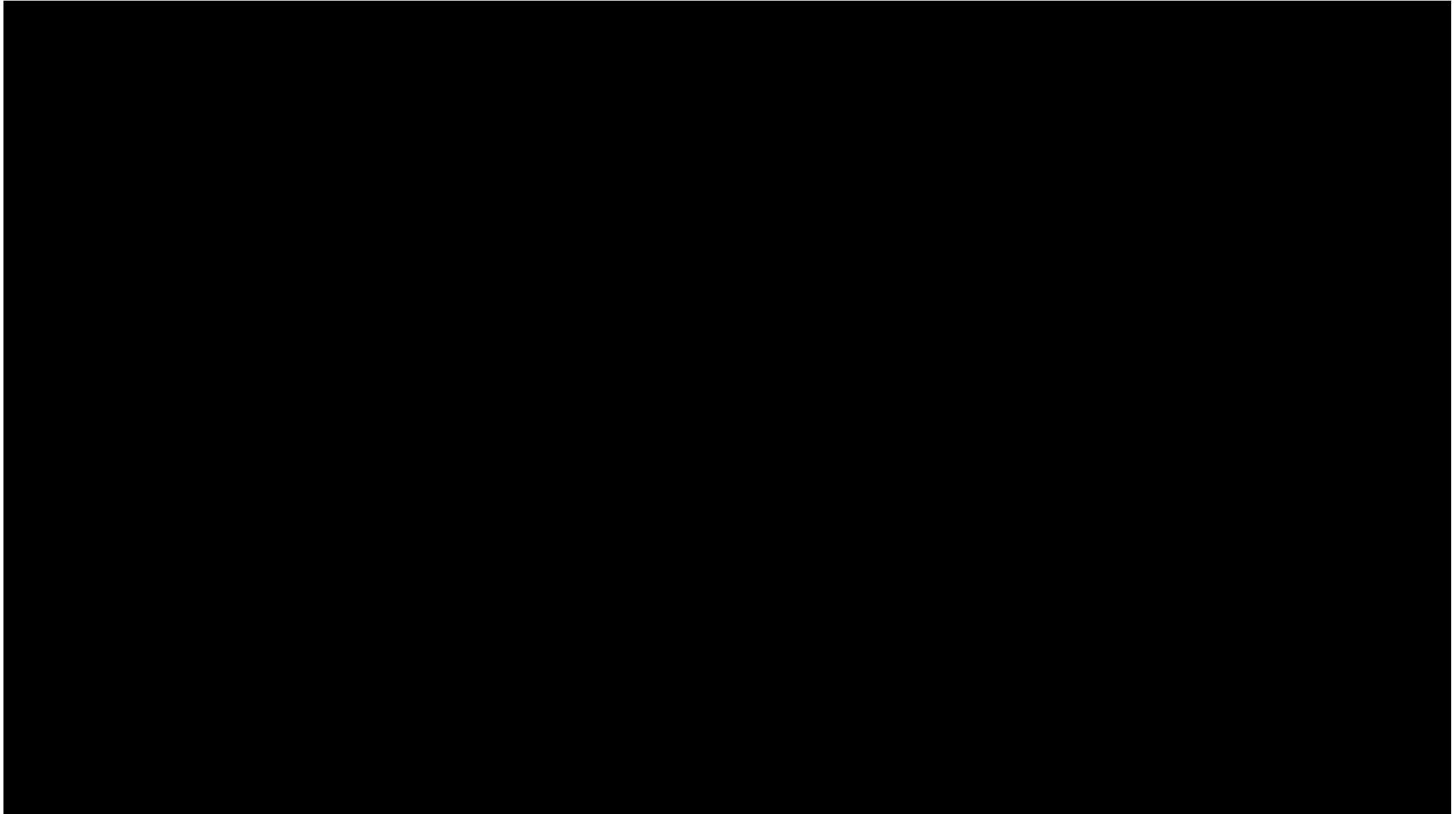
여기에 연구진은 사고 복제를 적용한 에이전트를 추가했다. 이는 앞선 에이전트와 달리, 행동을 결정하기 위해 시각 정보와 임무, 이에 따른 결정 사항 등을 '언어'로 재정리하도록 만든 것이다. 이를 통해 축적한 데이터로 다시 AI 에이전트를 재학습, 정확도를 끌어올린다는 의도다.

연구진은 복잡한 미로탈출 임무에서 사고 복제 AI 에이전트가 무려 80%의 성공률을 보였다고 전했다. 단순 행동 복제 에이전트는 30%에 그쳤다.

예를 들어 이미지나 비디오 입력이 가능한 'GPT-4 V'와 같은 모델에 내부 독백 구성 요소를 추가, 이를 통해 학습하는 내용을 더 유용하게 만들겠다고 밝혔다. 요리법이나 타이어 수리법, 포토샵으로 사진을 편집하는 법, 마인크래프트를 플레이하는 법 등에 적용, AI가 더 효과적인 솔루션을 찾아낼 수 있다는 말이다.

이에 대해 인지 및 언어학자인 안나 보르키 로마 사피엔자 대학교 심리학과 교수는 "가장 흥미로운 점은 언어의 존재가 AI에 유연성을 부여한다는 것"이라며 "복잡한 작업도 더 쉽게 수행할 수 있기 때문에 정말 효율적인 방식"이라고 밝혔다.

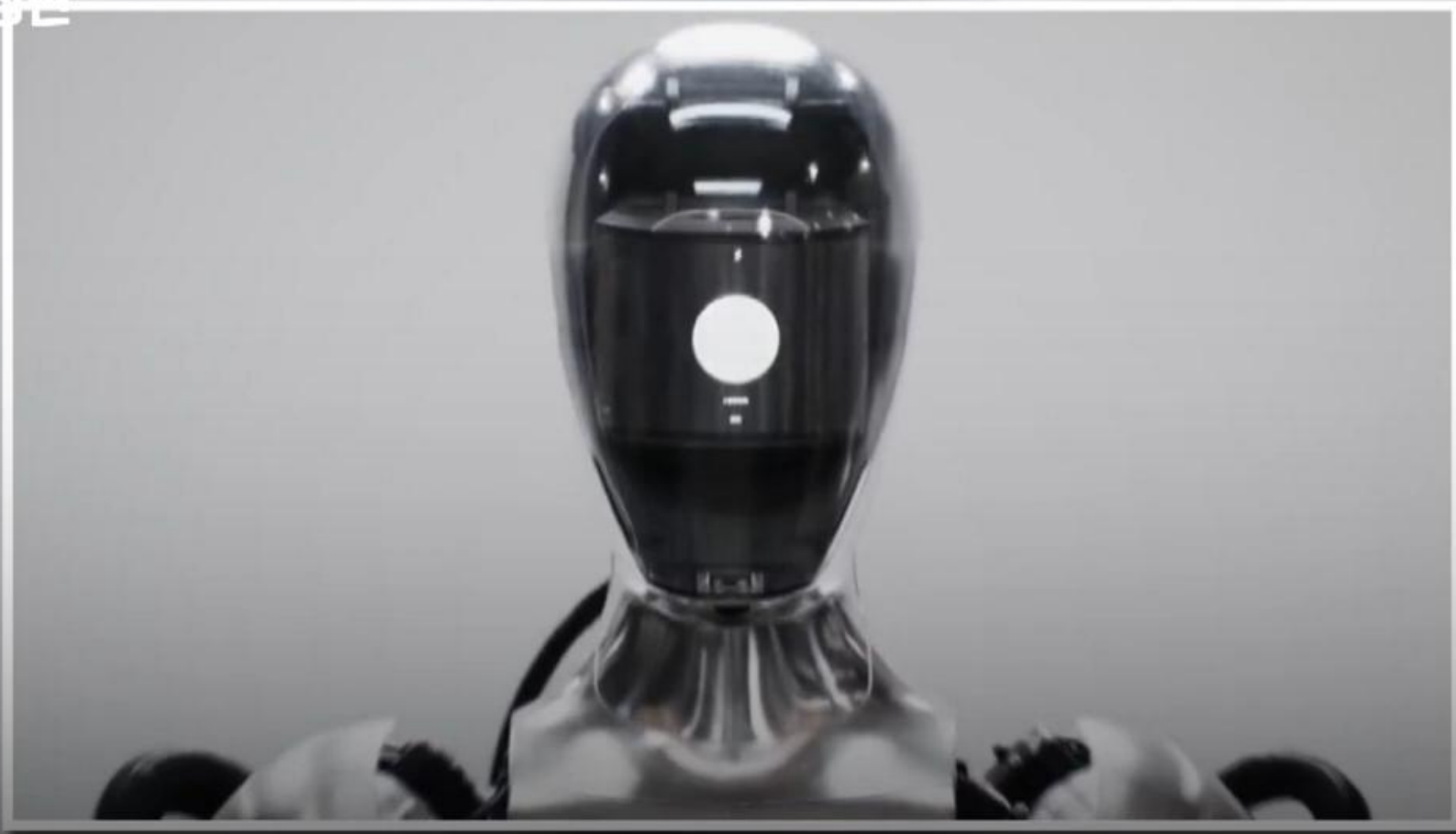
연구를 진행한 제프클론 브리티시 컬럼비아 대학교 컴퓨터 과학자는 "인간 지능의 강력한 사고 과정을 AI 에이전트에도 도입할 수 있다"라며 "독백은 인간에게는 단순한 작업이지만 AI에는 큰 도약이라고 할 수 있다"라고 말했다.





지금, 이 장면

jtbc



언어모델의 논리력

LLM의 약점 : 토큰화(Tokenizing) 문제(?)



다음 단어의 철자를 거꾸로 얘기해 줘.

lollipop

 "lollipop"의 철자를 반대로 하면 "poplli" 입니다.

popillol

Copy Retry < 2 / 2 > Share



Claude can make mistakes. Please double-check responses.

 lollipop의 철자를 거꾸로 하면 ppilolli입니다.

Copy Retry Share

 **뤼튼 GPT-3.5**

lollipop을 거꾸로 쓰면 pilpoli입니다.

 **뤼튼 PaLM2**

poppillol

다음 단어를 거꾸로 얘기해 줘.

간장공장

 "간장공장"을 거꾸로 말하면 "장공간장" 입니다.

장공간

Copy Retry Share



Claude can make mistakes. Please double-check responses.

 간장공장을 거꾸로 하면 "장공장간"입니다.

Copy Retry Share

 **뤼튼 GPT-3.5**

장간공간

 **뤼튼 PaLM2**

낭공장간

다시 생성 문장 다듬기

100원 200원 400원 500원 동전이 무한히 있다.
최소 동전 수로 800원 거스름돈을 주는 방법은?



800원을 거스름돈으로 주기 위해 최소 동전 수로 조합하는 방법은 다음과 같습니다.

- 500원 동전 1개와 100원 동전 3개를 사용하면 총 800원이 됩니다.

따라서, 최소 동전 수로 800원을 거스름돈으로 주기 위해서는 500원 동전 1개와 100원 동전 3개를 사용하는 것이 가장 효율적인 방법입니다.

논리력 테스트 (클로드3)



100원 200원 400원 500원 동전이 무한히 있다.
최소 동전 수로 800원 거스름돈을 주는 방법은?



Claude-3-Sonnet

Poe

네, 800원을 거슬러 줄 수 있는 다른 방법들도 있습니다. 예를 들면 다음과 같습니다:

1. 400원 동전 2개 사용
2. 200원 동전 4개 사용
3. 100원 동전 8개 사용
4. 500원 동전 1개, 200원 동전 1개, 100원 동전 1개 사용
5. 500원 동전 1개, 400원 동전 1개 사용
6. 500원 동전 1개, 200원 동전 2개 사용
7. 500원 동전 1개, 100원 동전 3개 사용

이렇게 다양한 방법으로 800원을 거슬러 줄 수 있지만, 앞서 말씀드린 500원 1개, 200원 1개를 사용하는 것이 가장 적은 수의 동전을 사용하는 최소 동전 수 방식입니다.

100원 200원 400원 500원 동전이 무한히 있다.
최소 동전 수로 800원 거스름돈을 주는 방법은?



동전이 무한히 있는 상황에서 최소 동전 수로 800원을 거슬러 주는 방법을 찾으려면, 가장 큰 금액의 동전부터 사용하면 됩니다.

1. 500원 동전 1개 사용: $800\text{원} - 500\text{원} = 300\text{원}$
2. 400원 동전 1개 사용: $300\text{원} - 400\text{원} = -100\text{원}$ (이 경우는 음수이므로 불가능합니다.)
3. 200원 동전 1개 사용: $300\text{원} - 200\text{원} = 100\text{원}$
4. 100원 동전 1개 사용: $100\text{원} - 100\text{원} = 0\text{원}$

따라서, 최소 동전 수로 800원을 거슬러 주는 방법은 500원 1개, 200원 1개, 100원 1개로 총 3개의 동전을 사용하는 것입니다.

논리력 테스트 (클로버 X)



최선을 다해 차근차근 다음의 형식을 이용해서 결과물을 작성해 주세요.

질문 : 입력된 질문
생각: 질문에 대한 다양한 해결법 생각.
수행 : 해결법을 하나씩 수행
관찰: 수행된 결과 관찰
... (이 생각/수행/관찰 은 N회 반복 가능)

최종 답변: 입력된 요청에 대한 최종 결과물

시작 ~~~

질문 : 100원 200원 400원 500원 동전이 무한히 있다.
최소 동전 수로 800원 거스름돈을 주는 방법은?



질문: 100원, 200원, 400원, 500원 동전이 무한히 있다. 최소 동전 수로 800원 거스름돈을 주는 방법은?

생각: 800원을 만들기 위해 사용할 수 있는 동전은 100원, 200원, 400원, 500원이다. 이 중에서 가장 적은 수의 동전을 사용하여 800원을 만들어야 한다.

수행: 먼저 500원 동전을 사용하여 최대한 많은 금액을 만든다. 500원 동전 2개를 사용하면 1000원이 된다. 이제 1000원에서 800원을 빼면 200원이 남는다. 남은 200원을 만들기 위해 100원 동전 2개를 사용한다.

관찰: 500원 동전 2개와 100원 동전 2개를 사용하여 총 4개의 동전으로 800원을 만들었다.

최종 답변: 최소 동전 수로 800원 거스름돈을 주는 방법은 500원 동전 2개와 100원 동전 2개를 사용하는 것입니다.

논리력 테스트 (클로드3)



최선을 다해 차근차근 다음의 형식을 이용해서 결과물을 작성해 주세요.

질문 : 입력된 질문
생각: 질문에 대한 다양한 해결법 생각.
수행 : 해결법을 하나씩 수행
관찰: 수행된 결과 관찰
... (이 생각/수행/관찰 은 N회 반복 가능)

최종 답변: 입력된 요청에 대한 최종 결과물

시작 ~~~

질문 : 100원 200원 400원 500원 동전이 무한히 있다.
최소 동전 수로 800원 거스름돈을 주는 방법은?

미련이 남은 듯 ~~~ ^^

Claude-3-Sonnet Poe

질문: 100원 200원 400원 500원 동전이 무한히 있다. 최소 동전 수로 800원 거스름돈을 주는 방법은?

생각: 이 문제를 해결하기 위해서는 동전의 조합을 바꿔서 다른 방법도 있나요?

동전의 조합을 바꿔서 다른 방법도 있나요?

Claude-3-Sonnet Poe

질문: 100원 200원 400원 500원 동전이 무한히 있다. 최소 동전 수로 800원 거스름돈을 주는 방법은?

생각: 앞서 제시한 500원 1개, 200원 1개, 100원 1개의 조합 외에 다른 조합도 가능할 것 같습니다.

수행: 400원 동전 2개를 사용해 봅니다.

관찰: 800원에서 400원 동전 2개(800원)를 뺀 나머지는 0원입니다. 이 경우 총 2개의 동전이 사용되었습니다.

생각: 400원 동전 2개를 사용하는 것이 가장 적은 동전 수인 것 같습니다. 하지만 500원 동전을 활용하는 방법도 있을 것입니다.

수행: 500원 동전 1개와 200원 동전 1개, 100원 동전 1개를 사용해 봅니다.

관찰: 500원 + 200원 + 100원 = 800원이 되며, 총 3개의 동전이 사용되었습니다.

최종 답변: 800원 거스름돈을 주는 최소 동전 수 방법은 400원 동전 2개를 사용하는 것입니다.

하지만 500원 1개, 200원 1개, 100원 1개의 조합도 가능하며, 이 경우 총 3개의 동전이 필요합니다.

논리력 테스트 (GPT-4o)



최선을 다해 차근차근 다음의 형식을 이용해서 결과물을 작성해 주세요.

질문 : 입력된 질문

생각: 질문에 대한 다양한 해결법 생각.

수행 : 해결법을 하나씩 수행

관찰: 수행된 결과 관찰

... (이 생각/수행/관찰 은 N회 반복 가능)

최종 답변: 입력된 요청에 대한 최종 결과물

시작 ~~~

질문 : 100원 200원 400원 500원 동전이 무한히 있다.
최소 동전 수로 800원 거스름돈을 주는 방법은?



질문: 100원, 200원, 400원, 500원 동전이 무한히 있다. 최소 동전 수로 800원 거스름돈을 주는 방법은?

생각: 먼저 800원을 만들기 위해 사용할 수 있는 동전들의 조합을 확인해보자. 각 동전의 가치가 100원, 200원, 400원, 500원이므로, 큰 동전부터 사용하여 최대한 적은 동전으로 800원을 만드는 방법을 찾아야 한다.

관찰:

1. 500원 동전 + 300원:

- 500원 + 200원 + 100원 = 800원, 총 3개의 동전이 사용된다.

2. 400원 동전 + 400원:

- 400원 + 400원 = 800원, 총 2개의 동전이 사용된다.

3. 200원 동전 + 600원:

- 200원 + 200원 + 400원 = 800원, 총 3개의 동전이 사용된다.

4. 100원 동전만 사용하여 800원:

- 100원 8개로 800원을 만들 수 있지만, 동전 수가 8개로 많아 최적의 방법이 아니다.

최종 답변: 최소 동전 수로 800원을 만드는 방법은 400원 동전 2개를 사용하는 것이다. 이 경우 총 2개의 동전만 필요하다.

논리력 테스트 : 숫자 크기



9.9 와 9.11 중에 큰 숫자는? 그리고, 그 이유를 설명하세요

 Claude-3-Sonnet [Poe](#)

9.11이 9.9보다 큰 숫자입니다.

소수점 뒤의 자리수가 클수록 숫자의 값이 커집니다. 9.9와 9.11 모두 정수 부분은 9로 같지만, 9.11의 소수 부분인 11이 9.9의 소수 부분 9보다 크기 때문에 전체 숫자 값은 9.11이 9.9보다 큼니다.



9.11이 9.9보다 큰 숫자입니다.

소수점 아래 숫자가 같을 경우, 소수점 위의 숫자가 클수록 큰 수입니다. 따라서 9.11이 9.9보다 큼니다.



[다른 답변 보기](#) 



9.11이 9.9보다 큰 숫자입니다.

이유는 소수점 이하의 값을 비교할 때, 먼저 소수점 앞의 숫자가 같으므로, 소수점 이하의 숫자를 비교하게 됩니다. 9.9는 소수점 이하가 0.9이고, 9.11은 소수점 이하가 0.11입니다. 0.11은 0.9보다 작지만, 두 자리 숫자에서 소수점 자리를 더 채우기 때문에 9.11이 9.9보다 큼니다.



 Gemini-1.5-Pro [Poe](#)

9.9 와 9.11 중에 큰 숫자는 **9.9** 입니다.

그 이유는 다음과 같습니다:

- **소수점 비교:** 두 숫자 모두 9라는 같은 정수 부분을 가지고 있습니다. 따라서 소수점 이하 부분을 비교해야 합니다.
- **소수 첫째 자리 비교:** 9.9는 소수 첫째 자리가 9이고, 9.11은 소수 첫째 자리가 1입니다. 9가 1보다 크기 때문에 9.9가 9.11보다 큼니다.

따라서 9.9가 9.11보다 큰 숫자입니다.

논리력 테스트 (GPT-4o)



최선을 다해 차근차근 다음의 형식을 이용해서 결과물을 작성해 주세요.

질문 : 입력된 질문
생각: 질문에 대한 다양한 해결법 생각.
수행 : 해결법을 하나씩 수행
관찰: 수행된 결과 관찰
... (이 생각/수행/관찰 은 N회 반복 가능)

최종 답변: 입력된 요청에 대한 최종 결과물

시작 ~~~

질문 : 9.9 와 9.11 중에 큰 숫자는? 근거와 함께 답해 줘

python 프로그램을 작성해서 해 봐

질문 : 9.9 와 9.11 중에 큰 숫자는? 근거와 함께 답해 줘

생각: 두 숫자 9.9와 9.11의 수학적 비교를 통해 큰 숫자를 찾아야 한다. 소수점 이하 자릿수가 중요한 역할을 할 것이다.

수행:

1. 9.9와 9.11을 직접 비교
2. 두 수의 소수점 첫 번째 자리를 확인하여 큰 숫자를 구별

관찰:

1. 9.9는 소수점 아래 첫 번째 자리에 9가 있고 9.11은 1이 있다.
2. 소수점 아래 첫 번째 자리를 비교하면 9.9가 더 크지만,

생각

최종

보다

python

Python으로 두 숫자 9.9와 9.11

항상 세부 정보 표시

코드 복사

입력된 숫자들

num1 = 9.9

num2 = 9.11

두 숫자 비교

larger number = max(num1, num2)



관찰: Python 프로그램을 통해 9.9와 9.11을 비교한 결과, 9.9가 더 큰 숫자로 출력되었습니다. 이는 부동소수점 계산의 문제일 수 있습니다.

최종 답변: Python의 부동소수점 계산 결과로 인해 9.9가 9.11보다 더 큰 숫자로 나왔지만, 이 결과는 잘못된 것일 수 있습니다. 수학적으로는 9.11이 더 큰 숫자입니다. [\[🔗\]](#)

최선을 다해 다음 질문의 가장 단순한 답하십시오.
다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
행동: 수행할 조치는 ['물 놓기', '물 버리기'] 중 하나여야 합니다.
행동 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

질문 :
비어있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?



ChatGPT

질문: 5리터 물을 얻는 가장 단순한 방법은 무엇입니까?
생각: 15리터 물병을 가득 채우고, 5리터 물병에 붓는 것이 가장 간단한 방법입니다.
행동: 물 놓기
관찰: 15리터 물병이 가득 찼습니다.
행동 입력: 물 놓기
관찰: 5리터 물병에는 5리터의 물이 들어 있습니다.
생각: 5리터 물을 얻었습니다.
최종 답변: 15리터 물병을 가득 채우고, 5리터 물병에 붓는 것이 가장 단순한 방법입니다.

최선을 다해 다음 질문의 가장 단순한 답하십시오.
다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
행동: 수행할 조치는 ['물 넣기', '물 버리기'] 중 하나여야 합니다.
행동 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

질문 :
비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?



생각: 비어있는 두 개의 물병이 있으므로, 이 두 물병을 이용하여 5리터의 물을 얻어야 한다.
행동: 먼저 15리터 물병에 물을 가득 채운다.
행동 입력: 물 넣기
관찰: 15리터 물병에 물이 가득 찼다.
생각: 이제 15리터 물병에서 5리터 물병으로 물을 옮긴다.
행동: 15리터 물병의 물을 5리터 물병으로 옮긴다.
행동 입력: 물 옮기기
관찰: 5리터 물병에 정확히 5리터의 물이 들어있다.
생각: 나는 이제 5리터의 물을 얻는 방법을 알았다.
최종 답변: 비어있는 5리터와 15리터 물병을 이용하여, 먼저 15리터 물병에 물을 가득 채운 후, 그 물을 5리터 물병으로 옮기면 5리터의 물을 얻을 수 있다.

최선을 다해 다음 질문의 가장 단순한 답하십시오.
다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
행동: 수행할 조치는 ['물 넣기', '물 버리기'] 중 하나여야 합니다.
행동 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

질문 :
비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?

질문: 비어 있는 5리터 물병과 15리터 물병이 있습니다. 5리터의 물을 얻는 방법은?

생각: 이 문제를 해결하기 위해서는 두 개의 물병을 사용하여 5리터의 물을 남겨야 합니다. 물을 채우고 비우는 일련의 단계가 필요할 것입니다.

행동: ['물 넣기']

행동 입력: 15리터 물병을 가득 채웁니다.

관찰: 15리터 물병이 가득 찼고, 5리터 물병은 비어있습니다.

행동: ['물 넣기']

행동 입력: 15리터 물병의 물을 5리터 물병에 부어 5리터 물병을 가득 채웁니다.

관찰: 5리터 물병이 가득 찼고, 15리터 물병에는 10리터의 물이 남아있습니다.

행동: ['물 버리기']

행동 입력: 5리터 물병의 물을 모두 버립니다.

관찰: 5리터 물병이 비었고, 15리터 물병에는 10리터의 물이 남아있습니다.

행동: ['물 넣기']

행동 입력: 15리터 물병의 남은 10리터 물을 5리터 물병에 부어 가득 채웁니다.

관찰: 5리터 물병이 가득 찼고, 15리터 물병이 비었습니다.

생각: 이제 5리터 물통에 정확히 5리터의 물이 있으므로 원하는 결과를 얻었습니다.

최종 답변: 비어있는 5리터 물병과 15리터 물병이 있을 때, 5리터의 물을 얻는 절차는 다음과 같습니다:

1. 15리터 물병을 가득 채웁니다.
2. 15리터 물병의 물을 5리터 물병에 부어 5리터 물병을 가득 채웁니다.
3. 5리터 물병의 물을 모두 버립니다.
4. 15리터 물병의 남은 10리터 물을 5리터 물병에 부어 가득 채웁니다. 이렇게 하면 5리터 물병에 정확히 5리터의 물이 남게 됩니다.

논리력 테스트 : 클로드3.5



최선을 다해 다음 질문의 가장 단순한 답하십시오.
다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야할지 항상 생각해야 합니다.
행동: 수행할 조치는 ['물 넣기', '물 버리기'] 중 하나여야 합니다.
행동 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

질문 :
비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?

질문: 비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 방법은?

생각: 15리터 물병에 물을 채운 다음 5리터 물병으로 옮기면 될 것 같습니다.

행동: 물 넣기

행동 입력: 15리터 물병

관찰: 15리터 물병이 가득 찼습니다.

생각: 이제 15리터에서 5리터로 물을 옮기면 됩니다.

행동: 물 버리기

행동 입력: 15리터 물병에서 5리터 물병으로

관찰: 5리터 물병에 정확히 5리터의 물이 있습니다.

생각: 이제 최종 답을 알았습니다.

최종 답변: 15리터 물병을 가득 채운 후, 그 물을 5리터 물병에 부으면 됩니다.

Copy Retry



Claude can make mistakes. Please double-check responses.

최선을 다해 다음 질문의 가장 단순한 답하십시오.
다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야 할지 항상 생각해야 합니다.
행동: 수행할 조치는 ['물 넣기', '물 버리기'] 중 하나여야 합니다.
행동 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

질문 :
비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?



질문: 비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?
생각: 5리터 물병에 정확히 5리터의 물을 채우려면, 물을 채우고 비우는 행동으로 목표를 달성할 수 있습니다.
행동: 물 넣기
행동 입력: 15리터 물병
관찰: 15리터 물병이 가득 찼습니다.
생각: 15리터 물병에서 5리터 물병에 물을 채우면 됩니다.
행동: 물 넣기
행동 입력: 5리터 물병 (15리터 물병에서 물을 따릅니다)
관찰: 5리터 물병이 가득 찼고, 15리터 물병에는 10리터가 남았습니다.
생각: 나는 이제 최종 답을 안다.
최종 답변: 15리터 물병에 물을 채운 후, 5리터 물병에 물을 부으면 정확히 5리터의 물을 얻을 수 있습니다.

최선을 다해 다음 질문의 가장 단순한 답하십시오.
다음 도구에 액세스할 수 있습니다.
다음 형식을 사용하십시오.

질문: 대답해야 하는 입력 질문
생각: 무엇을 해야할지 항상 생각해야 합니다.
행동: 수행할 조치는 ['물 넣기', '물 버리기'] 중 하나여야 합니다.
행동 입력: 액션에 대한 입력
관찰: 행동의 결과
... (이 생각/행동/행동 입력/관찰은 N회 반복 가능)
생각: 나는 이제 최종 답을 안다
최종 답변: 원래 입력된 질문에 대한 최종 답변

질문 :
비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?

✿ 답변

질문: 비어있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 방법은?

생각: 15리터 물병을 활용하여 5리터 물병에 정확히 5리터를 채우는 방법을 찾아야 합니다.

행동: 물 넣기

행동 입력: 15리터 물병에 물을 가득 채운다.

관찰: 15리터 물병이 가득 찼다.

행동: 물 넣기

행동 입력: 15리터 물병에서 5리터 물병에 물을 채운다.

관찰: 5리터 물병이 가득 찼고, 15리터 물병에는 10리터가 남았다.

행동: 물 버리기

행동 입력: 5리터 물병의 물을 모두 버린다.

관찰: 5리터 물병이 비었다.

행동: 물 넣기

행동 입력: 15리터 물병에서 5리터 물병에 다시 물을 채운다.

관찰: 5리터 물병이 다시 가득 찼고, 15리터 물병에는 정확히 5리터가 남았다.

생각: 이제 정확히 5리터의 물을 얻었다.

최종 답변: 위의 과정을 통해 정확히 5리터의 물을 얻을 수 있다.

[참고] o1 Reasoning Tokens (Slow Thinking)



Model	Pricing
o1	\$15.00 / 1M input tokens
	\$7.50 / 1M cached* input tokens
	\$60.00 / 1M output** tokens

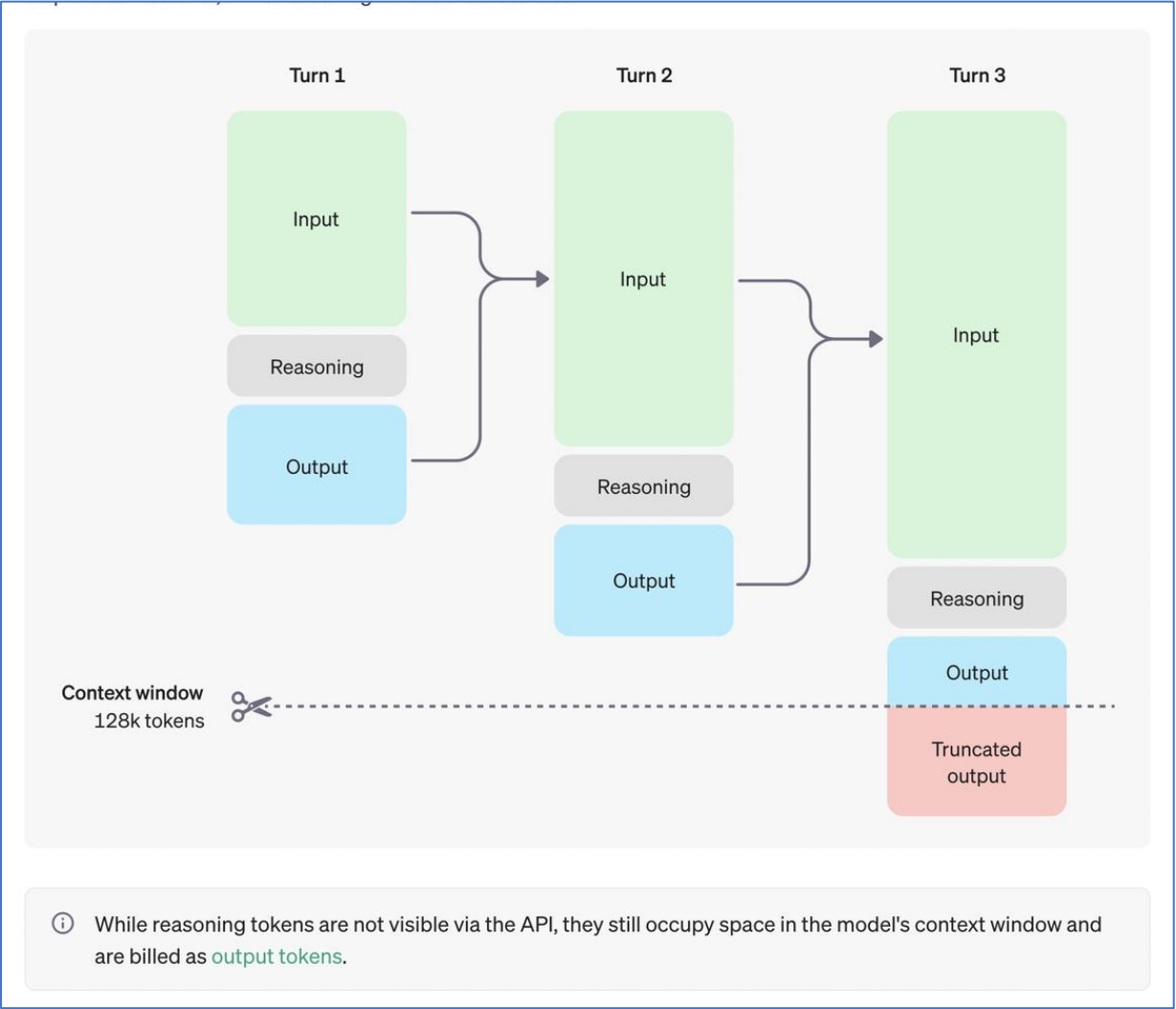
*Cached prompts are offered at a 50% discount compared to uncached prompts. [Learn more about Prompt Caching](#) ↗

**Output tokens include internal reasoning tokens generated by the model that are not visible in API responses.

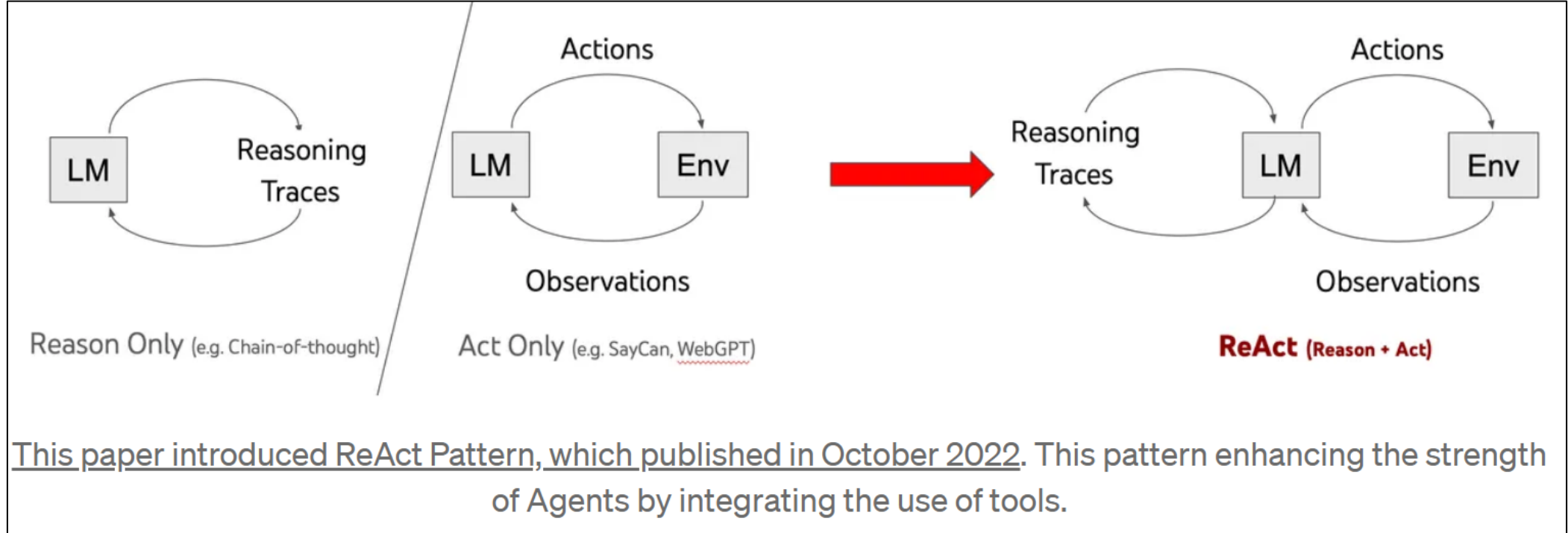
Prompt caching

Reduce latency and cost with prompt caching.

MODEL	TEXT INPUT COST
gpt-4o (excludes gpt-4o-2024-05-13 and chatgpt-4o-latest)	50% less
gpt-4o-mini	50% less
gpt-4o-realtime-preview	50% less
o1-preview	50% less
o1-mini	50% less



[참고] Reasoning을 위한 Prompt 기법 : ReAct



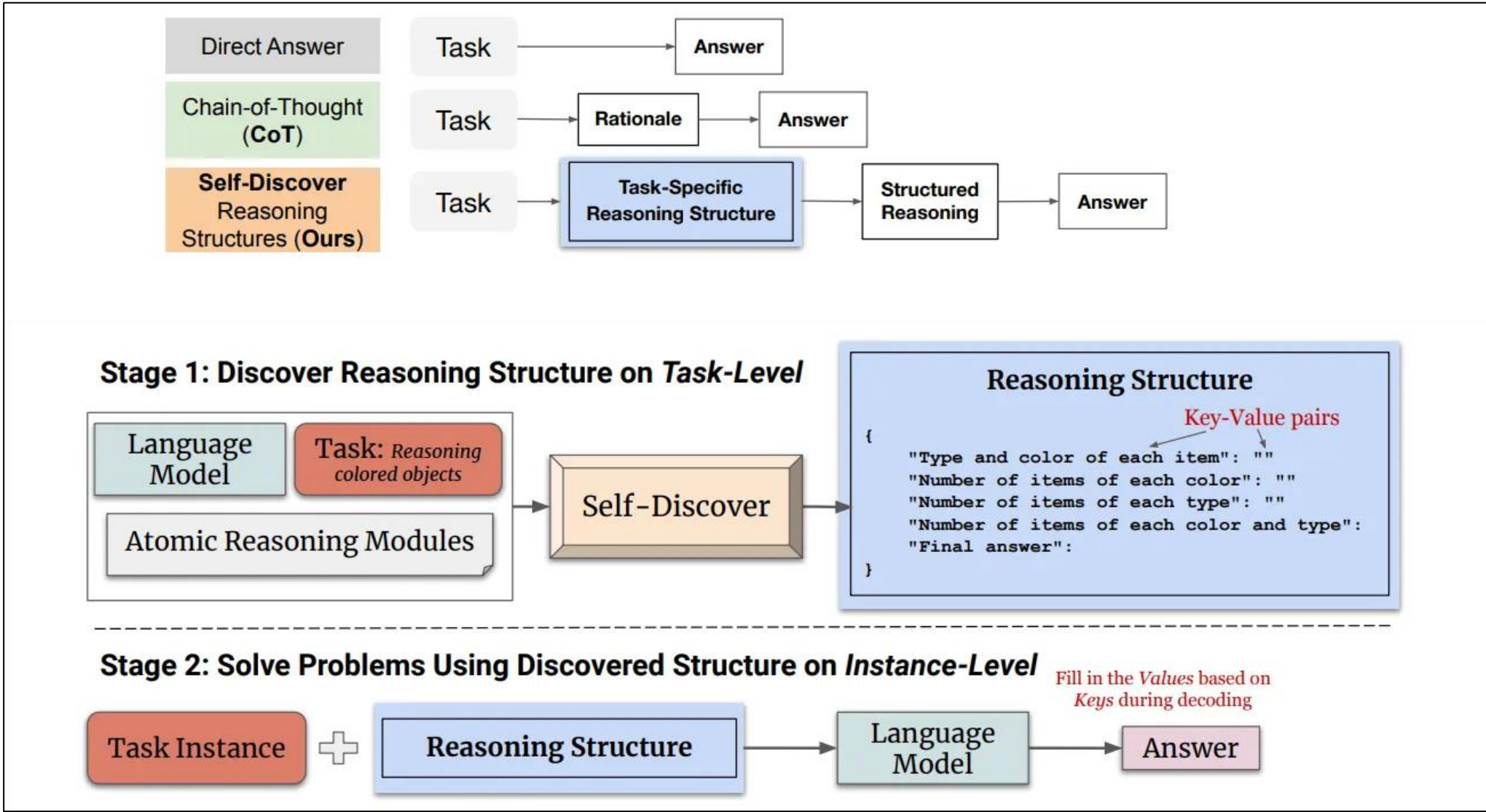
ReAct 구현 방법 링크 : <https://medium.com/binome/ai-agent-workflow-design-patterns-an-overview-cf9e1f609696>

[참고] Reasoning을 위한 Prompt 기법 : ReAct



Prompt_ID	Type	Trigger Sentence
101	CoT	Let's think step by step.
201	PS	Let's first understand the problem and devise a plan to solve the problem. Then, let's carry out the plan to solve the problem step by step.
301	PS+	Let's first understand the problem, extract relevant variables and their corresponding numerals, and devise a plan. Then, let's carry out the plan, calculate intermediate variables (pay attention to correct numeral calculation and commonsense), solve the problem step by step, and show the answer.
302	PS+	Let's first understand the problem, extract relevant variables and their corresponding numerals, and devise a complete plan. Then, let's carry out the plan, calculate intermediate variables (pay attention to correct numerical calculation and commonsense), solve the problem step by step, and show the answer.
303	PS+	Let's devise a plan and solve the problem step by step.
304	PS+	Let's first understand the problem and devise a complete plan. Then, let's carry out the plan and reason problem step by step. Every step answer the subquestion, "does the person flip and what is the coin's current state?". According to the coin's last state, give the final answer (pay attention to every flip and the coin's turning state).
305	PS+	Let's first understand the problem, extract relevant variables and their corresponding numerals, and make a complete plan. Then, let's carry out the plan, calculate intermediate variables (pay attention to correct numerical calculation and commonsense), solve the problem step by step, and show the answer.
306	PS+	Let's first prepare relevant information and make a plan. Then, let's answer the question step by step (pay attention to commonsense and logical coherence).
307	PS+	Let's first understand the problem, extract relevant variables and their corresponding numerals, and make and devise a complete plan. Then, let's carry out the plan, calculate intermediate variables (pay attention to correct numerical calculation and commonsense), solve the problem step by step, and show the answer.

Prompt_ID	유형	트리거 문장
101	CoT	하나씩 차근차근 생각해 봅시다.
201	PS	먼저 문제를 이해하고 문제를 해결하기 위한 계획을 세워 봅시다. 그런 다음 계획을 실행하여 문제를 단계적으로 해결해 봅시다.
301	PS+	먼저 문제를 이해하고, 관련 변수를 추출하며 해당 수치를 파악하고 계획을 세워 봅시다. 그런 다음 계획을 실행하고 중간 변수를 계산하여 (정확한 숫자 계산과 상식을 염두에 두고) 문제를 단계적으로 해결하고 답을 제시합니다.
302	PS+	먼저 문제를 이해하고, 관련 변수를 추출하며 해당 수치를 파악하고 완전한 계획을 세워 봅시다. 그런 다음 계획을 실행하고 중간 변수를 계산하여 (정확한 숫자 계산과 상식을 염두에 두고) 문제를 단계적으로 해결하고 답을 제시합니다.
303	PS+	계획을 세우고 문제를 단계적으로 해결해 봅시다.
304	PS+	먼저 문제를 이해하고, 완전한 계획을 세운 후 문제를 단계적으로 추론하여 해결해 봅시다. 각 단계에서 "사람이 동전을 뒤집는지 그리고 동전의 현재 상태는 무엇인지?"라는 하위 질문에 답하십시오. 동전의 마지막 상태에 따라 최종 답을 제시하십시오 (각 뒤집기와 동전의 전환 상태에 주의).
305	PS+	먼저 문제를 이해하고, 관련 변수를 추출하며 해당 수치를 파악하고 완전한 계획을 세워 봅시다. 그런 다음 계획을 실행하고 중간 변수를 계산하여 (정확한 숫자 계산과 상식을 염두에 두고) 문제를 단계적으로 해결하고 답을 제시합니다.
306	PS+	관련 정보를 준비하고 계획을 세워 봅시다. 그런 다음 상식과 논리적 일관성을 염두에 두고 문제를 단계적으로 해결해 봅시다.
307	PS+	먼저 문제를 이해하고, 관련 변수를 추출하며 해당 수치를 파악하고 완전한 계획을 세워 봅시다. 그런 다음 계획을 실행하고 중간 변수를 계산하여 (정확한 숫자 계산과 상식을 염두에 두고) 문제를 단계적으로 해결하고 답을 제시합니다.



논리력 테스트 : GPT-4o (Slow Thinking)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차에서 시도되지 않은 새로운 해결법 생각.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :
비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?

 질문

비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?

관찰 1차

- 목표 달성: 5리터 물병에 정확히 5리터 물이 들어 있다.
- 사용된 행동: 물병 채우기, 물 옮기기.

결과 1차

15리터 물병을 채운 후, 5리터 물병에 물을 옮겨 5리터를 확보했다.

생각 2차

1차 해결법이 단순하고 효율적이므로, 다른 대안이 필요하지 않다.

최종 답변

15리터 물병에 물을 가득 채운 뒤, 5리터 물병이 가득 찰 때까지 물을 옮기면 5리터 물병에 정확히 5리터 물을 얻을 수 있습니다.

논리력 테스트 : GPT-4o (Slow Thinking)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차 결과보다 단순한 방법 고려.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :
비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?



질문:

비어 있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?

관찰 1차

- 5리터 물병이 정확히 채워졌음.
- 과정이 명확하고 실행 가능.

수행 2차

1. 5리터 물병을 직접 채운다.

결

15

관찰 2차

- 5리터 물병이 정확히 채워짐.
- 1차보다 더욱 간단한 방법.

생

•

•

최종 답변

5리터 물병을 직접 채우면 가장 단순하게 5리터 물을 얻을 수 있습니다.



논리력 테스트 : 클로드3.5 (Slow Thinking)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차에서 시도되지 않은 새로운 해결법 생각.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :
비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?

질문: 비어 있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법은?

생각1차: 가장 간단한 방법을 생각해보자

- 15리터 물병에 물을 가득 채운 후 5리터 물병에 옮기기
- 5리터 물병에 직접 물을 채우기

수행1차: 15리터 물병에 물을 가득 채운 후 5리터 물병으로 옮기기

관찰1차:

- 15리터 물병이 가득 참
- 15리터에서 5리터 물병으로 옮기면 정확히 5리터가 옮겨짐 결과1차: 목표 달성 - 정확히 5리터를 얻을 수 있음

생각2차: 5리터 물병에 직접 물을 채우는 방법 검토

수행2차: 5리터 물병에 물을 직접 채우기

관찰2차:

- 5리터 물병이 가득 참
- 첫 번째 방법과 동일한 결과를 얻을 수 있음
- 하지만 더 직접적이고 단순한 방법임

최종 답변: 5리터 물병에 직접 물을 채우는 것이 가장 단순한 방법입니다. 15리터 물병을 사용할 필요가 없습니다.

Copy Retry



Claude can make mistakes. Please double-check responses.

논리력 테스트 : Claude 3.5 (Slow Thinking)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차 결과보다 단순한 방법 고려.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :

비어 있는 5리터 물병과 15리터 물병이 있습니다.

5리터 물을 얻는 가장 단순한 방법은?

질문 : 비어 있는 5리터 물병과 15리터 물병이 있을 때 5리터 물을 얻는 가장 단순한 방법

생각 1차 :

- 15리터 물병에 물을 채우고 5리터씩 덜어내는 방법
- 5리터 물병을 직접 채우는 방법

생각 2차 :

- 5리터 물병을 직접 채우는 것이 가장 단순할 것 같음
- 굳이 15리터 물병을 사용할 필요가 없음

수행 2차 :

1. 5리터 물병에 물을 직접 채움

관찰 2차 :

- 한 번의 동작으로 목표 달성
- 추가적인 물 옮기기나 계산이 필요 없음
- 가장 효율적이고 단순한 방법임

최종 답변 :

5리터 물병에 직접 물을 채우면 됩니다. 15리터 물병은 사용할 필요가 없습니다.



논리력 테스트 : Gemini-1.5 (Slow Thinking ... : o1 스타일)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차에서 시도되지 않은 새로운 해결법 생각.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :

비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?

다시 새로운 방법 생각해 보

Gemini-1.5-Pro Poe

질문 : 비어 있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법
은?

관찰 1차 :

1. 첫 번째 방법은 5리터를 얻을 수 있지만, 물을 옮기는 과정이 두 번 필요하고, 5리터 물
병을 비우는 과정이 추가로 필요합니다.

Gemini-1.5-Pro Poe

질문 : 비어 있는 5리터 물병과 15리터 물병이 있습니다. 5리터 물을 얻는 가장 단순한 방법
은? (새로운 방법 고려)

생각 1차 : 이전에 15리터 물병을 가득 채우는 방법을 생각했습니다. 이번에는 15리터 물병
을 가득 채우지 않고 5리터를 얻는 방법을 생각해봅니다. 하지만, 물의 양을 측정할 도구가
없으므로 15리터 물병에 정확히 특정량의 물을 넣는 것은 불가능합니다. 따라서, 15리터 물

생각 2차 : "5리터 물을 얻는다"는 목표에 집중합니다. 물병의 용량이 5리터이므로, 5리터 물
병에 물을 채우는 것이 핵심입니다. 물의 출처에 대한 제약은 없습니다.

수행 2차 : 5리터 물병을 물 source (수도꼭지, 강 등) 에서 직접 채웁니다.

관찰 2차 : 5리터 물병에 물이 가득 찼습니다. 5리터의 물을 얻었습니다.

결과 2차 : 5리터 물병을 직접 채우는 것이 가장 단순한 방법입니다.

최종 답변 : 5리터 물병을 물 source에서 직접 채웁니다. 이것이 가장 단순한 방법입니다. 15
리터 물병은 사용하지 않습니다.

논리력 테스트 : Mixtral-8x7B-Chat



비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?



Mixtral-8x7B-Chat

작성자  @fireworksai

3.1천명의 팔로워



Mixtral 8x7B Mixture-of-Experts model from Mistral AI fine-tuned for instruction following

Hosted by Fireworks.ai: <https://app.fireworks.ai/models/fireworks/mixtral-8x7b-instruct>

이 봇은 서버 봇입니다. @fireworksai에서 관리하는 서버에 의해 구동됩니다. [자세히 보기](#)

20 메시지당 · [자세히 보기](#)

공식

비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?



Mixtral-8x7B-Chat

Poe

5리터 물병을 fill (ience) 하는 것입니다.

"fill"은 "빠짐없이 채우다, 완전히 채우다"라는 뜻이며, 이 경우 5리터 물병을 가득 채우는 것을 의미합니다.

비어 있는 3리터, 4리터, 7리터 물병이 있습니다.
7리터 물을 얻는 가장 단순한 방법은?



Mixtral-8x7B-Chat

Poe

3리터 물병을 가득 채우고, 7리터 물병에 3리터 물병을 부운 다음, 다시 3리터 물병을 채우고, 7리터 물병에 3리터 물병의 물을 부으면 7리터의 물을 얻을 수 있습니다.

이전에 7리터 물병에 3리터의 물을 부었기 때문에, 7리터 물병이 이미 3리터 정도 가득 찼다고 가정합니다. 그러면 3리터 물병을 가득 채우고, 7리터 물병에 3리터 물병을 부으면 7리터 물병이 가득 찼 때까지 2리터의 공간밖에 없기 때문에 7리터 물병은 완전히 가득 찼 수 없습니다.

그러나, 3리터 물병을 가득 채운 다음 7리터 물병에 3리터 물병을 부었기 때문에, 7리터 물병에는 3리터의 물이 들어 있습니다. 이제 3리터 물병을 가득 채우고, 7리터 물병에 3리터 물병의 물을 부으면 7리터 물병이 가득 찼 수 있습니다. 이 과정에서 3리터 물병에는 물이 없지만, 7리터 물병에는 물이 7리터 정도 가득 찼기 때문에, 7리터 물병에 물을 부을 공간이 있습니다.

비어 있는 5리터 물병과 15리터 물병이 있습니다.
5리터 물을 얻는 가장 단순한 방법은?

-  비어 있는 5리터 물병에 5리터 물을 채우는 것이 가장 단순하고 효과적인 방법입니다.
- 15리터 물병을 사용하는 경우, 다음과 같은 방법들이 있을 수 있지만, 모두 추가적인 작업이 필요합니다:
1. 15리터 물병에 물을 가득 채운 후, 5리터 물병에 물을 옮겨 담습니다. 이 방법은 10리터의 물을 낭비하게 됩니다.
 2. 15리터 물병에 물을 반만 채운 후, 5리터 물병에 물을 옮겨 담고, 다시 15리터 물병에 물을 채워 반복합니다. 이 방법은 두 번의 작업이 필요하며, 시간이 더 소요됩니다.
 3. 15리터 물병에 물을 3/4 정도 채운 후, 5리터 물병에 물을 옮겨 담고, 다시 15리터 물병에 물을 채워 반복합니다. 이 방법도 두 번의 작업이 필요하며, 첫 번째 방법보다는 조금 더 효율적이지만 여전히 낭비되는 물이 존재합니다.
- 따라서, 비어 있는 5리터 물병에 직접 5리터 물을 채우는 것이 가장 간단하고 효율적인 방법입니다.

비어 있는 3리터, 4리터, 7리터 물병과 14리터 물병이 있습니다.
7리터 물을 얻는 가장 단순한 방법은?

 비어 있는 3리터와 7리터 물병을 이용하여 7리터의 물을 얻는 가장 단순한 방법은 다음과 같습니다:

1. 3리터 물병에 물을 가득 채웁니다.
2. 3리터 물병의 물을 7리터 물병에 옮겨 담습니다.
3. 7리터 물병에는 총 7리터의 물이 담겨 있게 됩니다.

이 방법은 한 번의 작업으로 7리터의 물을 얻을 수 있으며, 추가적인 물 낭비나 복잡한 과정 없이 간단하게 수행할 수 있습니다.

마지막으로, 14리터 물병을 사용하는 경우, 다음과 같은 방법이 가능합니다:

1. 14리터 물병에 물을 가득 채웁니다.
2. 7리터 물병에 물을 옮겨 담습니다.
3. 7리터 물병에는 총 7리터의 물이 담겨 있게 됩니다.

이 방법은 한 번의 작업으로 7리터의 물을 얻을 수 있지만, 14리터의 물을 낭비하게 되므로 비효율적입니다.

따라서, 비어 있는 3리터와 7리터 물병을 이용하여 7리터의 물을 얻는 것이 가장 단순하고 효율적인 방법입니다.

논리력 테스트 : Clova-X (Slow Thinking)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차에서 시도되지 않은 새로운 해결법 생각.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :

비어 있는 5리터 물병과 15리터 물병이 있습니다.

5리터 물을 얻는 가장 단순한 방법은?



생각 1차: 비어있는 5리터와 15리터 물병을 이용하여 5리터의 물을 채우는 방법을 생각해봅니다. 먼저 15리터 물병에 물을 가득 채운 후, 이를 5리터 물병에 부어봅니다.

수행 1차: 15리터 물병에 물을 가득 채운 후, 이를 5리터 물병에 붓습니다. 그러면 5리터 물병에는 5리터의 물이 차고, 15리터 물병에는 10리터의 물이 남게 됩니다.

관찰 1차: 5리터 물병에 정확히 5리터의 물이 찼으므로 목표를 달성했습니다.

결과 1차: 15리터 물병을 이용하여 5리터의 물을 얻었습니다.

생각 2차: 다른 방법으로 5리터의 물을 얻을 수 있는지 확인합니다.

수행 2차: 이번에는 5리터 물병에 직접 물을 가득 채워봅니다.

관찰 2차: 5리터 물병에 직접 물을 가득 채웠으므로 또 다른 방법도 성공했습니다.

최종 답변: 비어있는 5리터와 15리터 물병 중 15리터 물병을 이용하여 5리터의 물을 얻거나, 5리터 물병에 직접 물을 가득 채워 5리터의 물을 얻을 수 있습니다.

논리력 테스트 : Clova-X (Slow Thinking)



차근차근 주어진 질문에 답하십시오.
다음 형식을 사용하십시오.

질문 : 입력된 질문

생각 1차 : 질문에 대한 다양한 해결법 생각.

수행 1차 : 해결법을 하나씩 수행

관찰 1차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

결과 1차 : 앞의 시도에서 나온 결과

생각 2차 : 1차에서 시도되지 않은 새로운 해결법 생각.

수행 2차 : 해결법을 하나씩 수행

관찰 2차 : 수행된 결과와 목표 비교 관찰

... (위의 생각/수행/관찰 은 N회 반복 가능)

최종 답변 : 입력된 요청에 대한 최종 결과물

질문 :

비어 있는 3리터, 4리터, 7리터 물병과 14리터 물병이 있습니다.
7리터 물을 얻는 가장 단순한 방법은?



생각 1차: 비어있는 3리터, 4리터, 7리터, 14리터 물병을 이용하여 7리터의 물을 채우는 방법을 생각해봅니다. 먼저 7리터 물병에 물을 가득 채운 후, 이를 사용합니다.

수행 1차: 7리터 물병에 물을 가득 채운 후, 이를 사용합니다.

관찰 1차: 7리터 물병에 이미 7리터의 물이 차 있으므로 바로 사용할 수 있습니다.

결과 1차: 7리터 물병을 이용하여 7리터의 물을 얻었습니다.

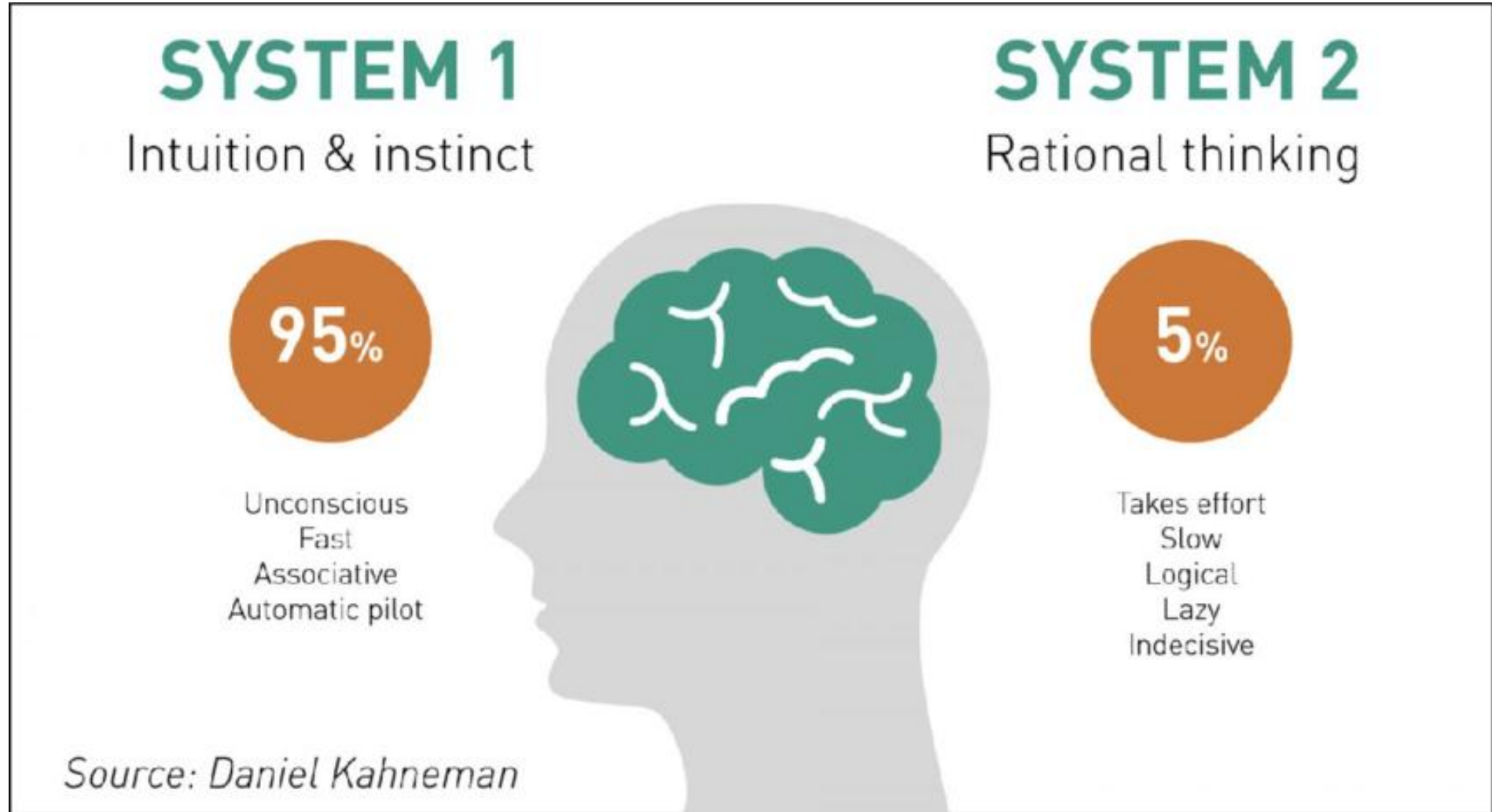
생각 2차: 다른 방법으로 7리터의 물을 얻을 수 있는지 확인합니다.

수행 2차: 이번에는 3리터와 4리터 물병을 이용하여 7리터를 채워봅니다. 먼저 3리터 물병에 물을 가득 채운 후, 이를 4리터 물병에 붓습니다. 그러면 4리터 물병에는 $3+4=7$ 리터의 물이 차게 됩니다.

관찰 2차: 3리터와 4리터 물병을 이용하여 7리터의 물을 얻었으므로 또 다른 방법도 성공했습니다.

최종 답변: 비어있는 3리터, 4리터, 7리터, 14리터 물병 중 7리터 물병을 이용하여 7리터의 물을 얻거나, 3리터와 4리터 물병을 이용하여 7리터의 물을 얻을 수 있습니다.





Founders Need to Think Slow, Move Fast

Discover the Data — React With Your Gut.

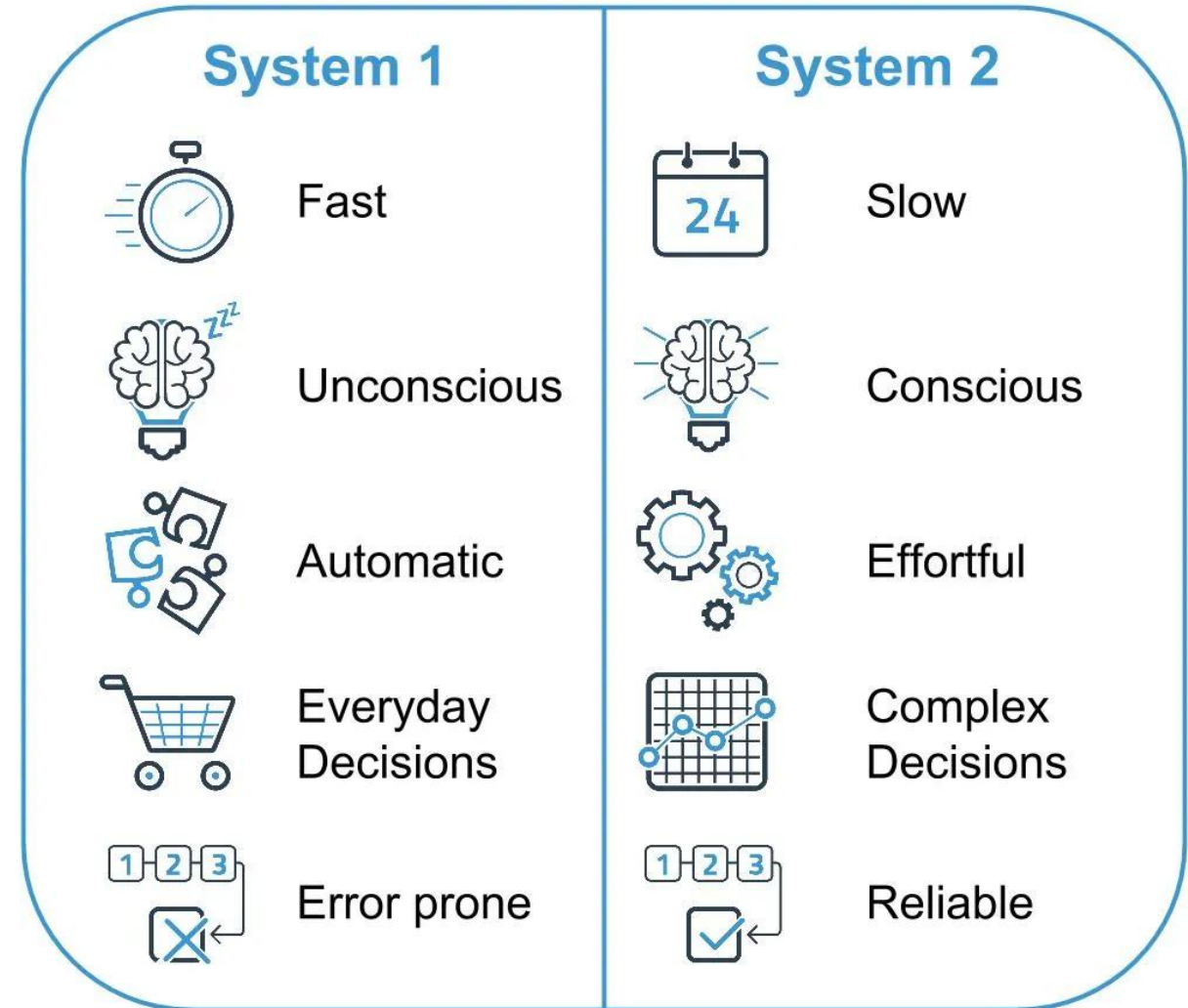


Ryan Sheffer · Follow
4 min read · Dec 18, 2015

The Data Isn't Enough

Once you have the data about your failures, it's not enough. Having a firm grip on the market or product need is incredibly valuable, but only if it's acted upon. Earlier, I brought up that the System 2 brain is great at analysis but it lacks the skills of instant action that the intuitive System 1 allows for. Our goal as founders is not to be deceived by our own intuition but instead run deep analysis. And we must act once the data is in. If you are spending your time understanding the market, you're not creating it.

As founders, we must strive for this level of deep analysis, lack of fear in the face of stories that say we have no chance, and instantaneous reaction when we know we've figured it out.



창업자는 천천히 생각하고 빠르게 움직여야 합니다.

데이터를 발견하고 직감적으로 반응하십시오.



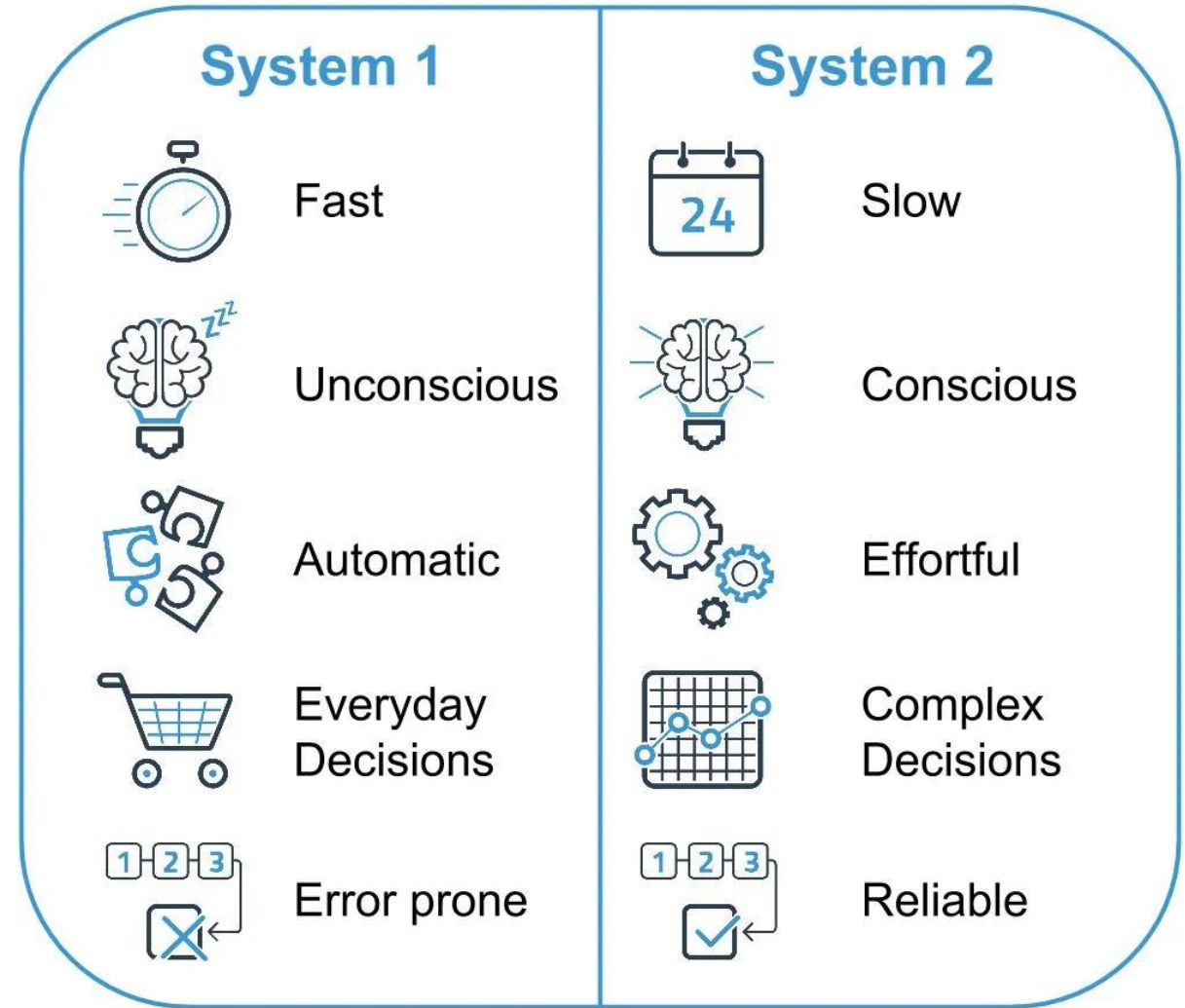
라이언 셰퍼(Ryan Sheffer) · 뒤따르다

4분 분량의 글 · 2015년 12월 18일

데이터만으로는 충분하지 않습니다

실패에 대한 데이터를 확보한 후에는 이것만으로는 충분하지 않습니다. 시장이나 제품의 요구 사항을 확실하게 파악하는 것은 매우 중요하지만 행동에 나서야 합니다. 앞서 나가는 시스템 2의 두뇌가 분석에는 뛰어나지만 직관적인 시스템 1이 허용하는 즉각적인 행동 능력이 부족하다고 지적했다. 창업자로서 우리의 목표는 우리 자신의 직관에 속지 않고 대신 깊은 분석을 실행하는 것입니다. 그리고 데이터가 들어오면 조치를 취해야 합니다. 시장을 이해하는 데 시간을 할애하고 있다면 시장을 창조하는 것이 아닙니다.

창업자로서 우리는 이러한 수준의 심층적인 분석, 우리에게 기회가 없다고 말하는 이야기에 직면했을 때 두려움을 느끼지 않고, 우리가 그것을 알아냈다는 것을 알았을 때 즉각적으로 반응하기 위해 노력해야 합니다.



알트먼 "AGI 구축법 이미 알아내...새로운 목표는 초지능 달성"

✎ 박찬 기자 ⌚ 입력 2025.01.07 18:05 ⌚ 수정 2025.01.08 07:32

샘 알트먼 오픈AI CEO가 인공지능(AGI)을 넘어 초지능(super intelligence) 개발에 나선다고 선언했다. 새해 들어 3번째 등장한 이야기로, 이제는 AGI 달성을 기정사실화하고 목표를 상향 조정한다는 것을 공식화하려는 것으로 보인다.

샘 알트먼 오픈AI CEO는 6일(현지시간) 개인 블로그를 통해 이제 목표를 초지능 개발로 전환하고 있다고 밝혔다.

이 글은 전날 블룸버그와의 인터뷰를 통해 오픈AI의 설립과 챗GPT 출시, 이사회로부터의 축출 등 지난 10년여 간의 여정에 대해 밝힌 데 이어 등장한 것이다. 인터뷰 내용을 보충하려는 시도로 보인다.

이 가운데 가장 눈길을 끈 부분은 역시 AGI와 초지능, 즉 ASI에 대한 것이다.

알트먼 CEO는 "우리는 이제 전통적으로 이해했던 대로 AGI를 구축하는 방법을 알고 있다고 확신한다"라고 밝혔다.

이어 "우리는 2025년에 최초의 AI 에이전트가 인력에 합류, 회사의 결과를 실질적으로 바꿀 수 있을 것이라고 믿는다"라고 설명했다. 즉, 현재 달성한 추론 모델의 성능이 에이전트 시스템과 결합하면, 인간 수준의 능력을 발휘할 것이라는 말이다.

알트먼 CEO는 전날 인터뷰에서도 이런 점을 강조한 바 있다. 그는 "'A가 매우 숙련된 인간이 할 수 있는 일을 할 수 있을 때가 AGI에 도달한 것'이라고 정의했다. 이미 o1 등 추론 모델로 특정 분야에서 인간의 지식을 넘어서 모델이 에이전트 시스템을 통해 자율성을 갖추고 숙련된 인간 수준으로 업무를 처리할 것으로 본 것이다. 이를 AGI를 구축하는 방법이라고 설명한 셈이다.

그리고 "우리는 그 이상의 목표를 진정한 의미의 초지능 달성으로 전환하기 시작했다"라고 강조했다.

그는 "초지능이 있으면, 우리는 무엇이든 할 수 있다. 초지능 도구는 우리가 스스로 할 수 있는 것보다 훨씬 더 과학적 발견과 혁신을 엄청나게 가속할 수 있으며, 큰 풍요와 번영을 이룰 수 있다"라고 말했다.

이번 발언은 지난해부터 이어진 AGI 달성이 가까워졌다는 말의 연장이다. 특히 지난 5일 X(트위터)를 통해 "특이점이 가까워졌다"라고 밝힌 데 이어, 블룸버그 인터뷰와 개인 블로그 글까지 동원해 AGI 달성을 기정 사실화하려는 분위기다.

한편, 이날 게시물과 블룸버그 인터뷰 중 눈에 띄는 다른 부분은 "AI 위험이 현실화될 수 있지만, 이를 해결하는 가장 좋은 방법은 제품을 출시하고 경험을 통해 배우는 것"이라고 주장한 내용이다, 이는 이전부터 나세웠던 논리로, 사전에 아무리 많은 안전 테스트를 거쳐도 완벽할 수는 없기 때문에 모델 출시를 지연하는 것이 답은 아니라는 말이다.

그는 또 일론 머스크가 트럼프 행정부에서 자신을 괴롭힐 가능성에 대한 질문에는 "그럴 것 같지 않다"라고 부정했다.

이 외에도 오픈AI 이사회가 자신을 해임했을 당시 가장 충격받은 순간으로 이사회가 에밋 시어를 CEO로 임명했던 때를 꼽았다. 그는 "그 순간 진짜 모든 것이 끝난 줄 알았다. 그것은 정말 충격적이었다"라고 회고했다.

메타 "AGI 도약 방법 발견... '트랜스포머'와 다른 아키텍처 개발 중"

✎ 임대준 기자 Ⓜ 입력 2024.04.10 12:48 📖 댓글 0 ♡ 좋아요 14

'AI 사대천왕' 중 하나이자 메타의 수석 AI 과학자인 안 르쿤이 생성 인공지능(AI)의 한계를 넘는 새로운 아키텍처를 개발 중이라고 밝혔다. 학습한 지식을 확률적으로 출력하는 현재의 '트랜스포머' 방식을 넘어, 사람처럼 추론하고 계획하는 방식으로 인공일반지능(AGI)에 도달할 수 있다고 설명했다.

테크크런치와 파이낸셜타임스 등은 9일(현지시간) 안 르쿤 수석이 런던에서 열린 '메타 AI 데이' 행사에 참여, 현재 생성 AI의 기반인 '트랜스포머'와는 다른 새로운 아키텍처 개발 중이라는 사실을 공개했다고 보도했다.

이에 따르면 르쿤 수석이 소개한 아키텍처는 '제파(JEPA, Joint Embedding Predicting Architecture)'라는 방식이다.

그는 현재 트랜스포머 기반 AI 시스템은 "실제로 생각이나 계획 없이 한 단어씩 차례로 생성"하는 구조라고 설명했다. 또 복잡한 질문을 다루거나 정보를 장기간 유지하는 데 어려움을 겪기 때문에 여전히 "어리석은 실수를 저지른다"라고 지적했다.

반면 제파는 대형언어모델(LLM)처럼 학습한 내용을 확률에 따라 뱉어내는 방식이 아닌, 실제로 추론하고 계획하는 방법이다.

"추론을 추가한다는 것은 AI 모델이 가능한 답을 검색하고 행동의 순서를 계획하며 행동의 효과가 어떻게 될지를 종합하는 '정신적 모델'을 구축한다는 것을 의미한다"라고 설명했다.

이미 이미지 생성 영역에서 정확한 답을 내기 위해 이를 활용했다고 밝혔다.



안 르쿤 메타 AI 수석 과학자 (사진=X)

또 이 방식은 AI 에이전트에 큰 도움이 될 수 있다고 밝혔다. 실제로 "파리에서 뉴욕 사무실까지 여행의 각 단계를 계획하고 예약하는 AI 에이전트를 개발 중"이라고 소개했다.

나아가 추론과 계획 능력은 결국 인공일반지능(AGI) 개발과도 연결된다고 강조했다.

르쿤 수석은 "이는 기계가 다음 단계의 지능을 달성하기 위해 노력하는 가운데 모두가 놓치고 있었던 중요한 부분"이라며 "AI의 미래는 생성 AI가 아니라 제파"라고 강조했다.

르쿤 수석은 최근 들어 AGI로 가기 위해서는 트랜스포머를 넘는 다른 방식이 필요하다고 수차례 강조해 왔다.

또 마크 저커버그 메타 CEO는 올 초 막대한 인프라를 구축, '라마 3'와 차세대 모델을 개발한다고 밝힌 바 있다. 저커버그 역시 올해부터 AGI 개발이 목표라고 밝혔다.

이는 르쿤 수석의 발언과 시기와 맞아 떨어진다. 즉 지난해 말쯤 중요한 발전을 이룬 것으로 볼 수 있다.

한편, 오픈AI 역시 '인간과 같은 추론 능력'을 AGI 개발의 핵심으로 꼽고 있다.

파이낸셜타임스에 따르면 브래드 라이트캡 오픈AI COO는 최근 인터뷰를 통해 "GPT-5는 이전 모델보다 더 많은 이야기를 할 것"이라며 "새 모델은 시간이 지남에 따라 스스로 생각하며, 더 길고 복잡한 작업을 수행하는 것을 보게 될 것"이라고 말했다. 이어 "그리고 이는 암묵적으로 추론 능력의 향상을 요구한다"라고 밝혔다.

이에 앞서 오픈AI는 지난해 말 '인간처럼 생각하는' 능력으로 정답이 확실한 수학 문제 등을 풀어내는 '큐스타'라는 모델을 개발, AGI 개발로 가는 돌파구를 찾은 것으로 알려졌다. 이 또한 수학적 추론 능력의 향상을 의미한다.

여기에 오픈AI도 이후 AI 에이전트를 개발 중인 것으로 알려졌다. 이마저도 메타와 흡사한 모습이다.

中 AI 벤처의 추론 모델..."오픈AI o1 성능 능가"

기사입력 : 2025년01월22일 09:37 | 최종수정 : 2025년01월22일 09:37

[베이징=뉴스핌] 조용성 특파원 = 중국의 AI(인공지능) 스타트업인 딥시크(DeepSeek, 중국명 선두추쉬, 深度求索)가 오픈AI의 o1에 버금가는 추론형 AI 모델을 발표했다.

딥시크가 21일 추론형 AI 모델인 딥시크-R1을 발표했다고 중국 매체 차이렌서(财联社)가 22일 전했다.

딥시크는 R1 모델이 주요 AI 벤치마크인 AIME와 MATH-500 지표에서 오픈AI의 추론형 AI 모델인 o1보다 높은 성능을 보였다고 소개했다. 또한 딥시크는 R1의 이용 가격이 o1보다 90% 이상 저렴하다고 덧붙였다.

대표적인 대화형 AI 대형 모델(LLM)인 챗GPT가 일반인들과의 대화를 목적으로 만들어졌다면, 추론형 AI 모델인 o1은 연구자나 전문직 종사자를 대상으로 수학, 과학, 코딩 등 복잡한 문제 해결 및 고급 추론 작업을 위해 만들어졌다.

딥시크의 R1은 오픈AI의 o1과 비슷한 대형 모델이다. 6710억 개의 매개변수를 포함하고 있으며, 복잡한 문제 해결에 필요한 방대한 데이터를 처리할 수 있다.

이에 앞서 딥시크는 지난해 12월 말 대화형 AI 대형 모델인 V3를 출시해 업계를 놀라게 했다. 출시 당시 딥시크는 "V3는 코딩 벤치마크(성능 평가)에서 오픈AI의 'GPT-4o', 메타의 '라마 3.1'을 넘어섰다"고 발표했다.

출처 : <https://www.newspim.com/news/view/20250122000227>

질문 : 100원 200원 400원 500원 동전이 무한히 있다. 최소 동전 수로 800원 거스름돈을 주는 방법은?

<생각>
또는 2
따라서
의 해결
잠깐만
그러면
아니면
총 3개
아니면
수도 있
400원
다른 것
요? 예

아니면 $400 + 400 = 800$, 즉 2개의 동전입니다.

네, 가장 효율적인 방법인 듯합니다.

또 다른 생각: 200원 동전 3개와 100원 동전 2개를 사용하면 어떨까요?
그러면 $600 + 200 = 800$ 개가 되지만, 5개로 2개보다 나쁩니다.

따라서 400원 동전 2개를 사용하는 것이 확실히 더 나은 선택입니다.

따라서 결론은 400원 동전 2개를 사용하는 것이 800원을 만드는 데 필요한 최소 동전 수라는 것입니다.

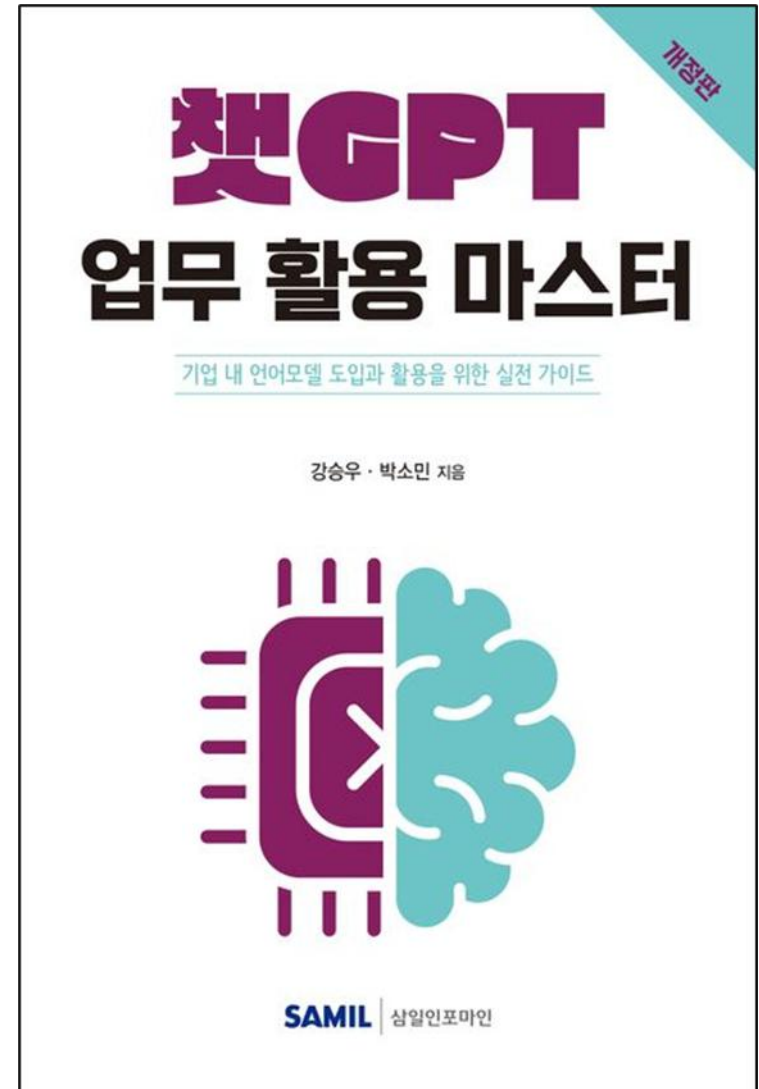
</think>

사용 가능한 액면가(100, 200, 400, 500)를 사용하여 800원을 만드는 데 필요한 최소 동전 수는 2개입니다. 이것은 400원 동전 2개를 사용하여 달성됩니다.

답: 400원 동전 2개를 사용하여 800원을 만들면 최소 동전 수(2)가 됩니다.

요약

- 임베딩 기술 : 자연어와 기계언어 통역 기술
- 자연어를 이해하는 ChatGPT
 - ✓ 자연어 기반 프로그래밍
- 추론 능력으로 발현되는 NLU
- 언어 모델의 약점 : 추론 → Prompt Engineering
 - ✓ Slow Thinking
 - ✓ 생각을 유도하는 CoT
- 성능과 정확도를 높이기 위한 다양한 시도
- 현실화 되는 지능화된 로봇
 - ✓ LLM + HW Robot



감사합니다.